

OS Lab 3: Process Scheduling

CS24BT032 – Anushaa B
CS24BT031 – Thiaana Balasani

1. What the lab asks us to investigate

The assignment is essentially an experiment in process scheduling. We build a scheduler simulator that reads the supplied workload descriptions, produces the schedule, and reports average turnaround time, maximum turnaround time, and simulator run time. Part I uses one processor; Part II repeats the study with two processors. The required policies are FIFO, Round Robin with different time quanta, and a three-level MLFQ with a comparison between no periodic boost and a boost every 20 time units.

Rather than treating the graphs as a collection of outputs, we use the report to connect three things: the structure of a workload, the decisions made by the scheduler, and the resulting turnaround times. That connection is particularly useful here because the same scheduling policy behaves very differently on different workloads.

2. Workloads

We used the five workload files supplied with our implementation: `process1.txt` through `process5.txt`.

Workload	Main characteristic
P1	CPU-only workload with long bursts and a later arrival.
P2	Repeated CPU-I/O cycles with comparatively long CPU bursts.
P3	Very short 1-unit CPU bursts separated by 5-unit I/O bursts.
P4	Mixed CPU-burst lengths with repeated I/O, giving the schedulers more opportunities to differ.
P5	A mixed workload with long CPU work together with highly bursty processes; particularly useful for RR and MLFQ parameter experiments.

The workload structure is important throughout the report. For example, a second processor can only help when there is enough independent CPU work to run in parallel; it cannot shorten an I/O burst. Similarly, changing an RR quantum matters much more when CPU bursts are long enough to be split into several turns.

3. Implementation: what the simulator actually keeps track of

Each process is represented by its arrival time, its complete CPU/I/O burst sequence, the current burst index, remaining time in the current burst, and completion time. RR additionally keeps a per-turn quantum counter. MLFQ keeps a queue level, a per-turn quantum counter, and a cumulative allotment counter.

The simulation advances one unit of simulated time at a time. At each tick it processes arrivals and I/O completions, applies an MLFQ priority boost when the configured interval is reached, performs

MLFQ priority preemption where necessary, dispatches work to idle CPUs, executes one unit on each running CPU, and finally handles completion, I/O blocking, quantum expiration and MLFQ demotion.

Turnaround time is computed as

$$T_i = C_i - A_i,$$

where A_i is arrival time and C_i is completion time.

The simulator runtime is measured around the simulation itself rather than around input/output, matching the assignment’s requirement that I/O should not be counted in the reported simulator run time.

A detail that matters for MLFQ

Our MLFQ uses three levels, Q_0 , Q_1 and Q_2 , with Q_0 having the highest priority and a fixed quantum of 2 time units. The implementation also tracks cumulative allotment at a level. A quantum counter resets when a new turn begins, while the allotment counter persists across an I/O block until demotion or a priority boost.

For the required experiment we use an allotment of 2, so exhausting the allotment causes a demotion after the corresponding quantum has been consumed. We separately vary the allotment later as an additional experiment.

At a priority boost, unfinished processes are returned to Q_0 and their MLFQ counters are reset. This includes processes currently waiting in the lower queues, currently running processes, and processes blocked for I/O. The last case is easy to miss: an I/O-blocked process is still unfinished, so its priority state must also be updated by a periodic boost.

4. Part I – One processor

4.1 FIFO baseline

FIFO gives us the simplest reference point because there is no time quantum and no priority-based reordering. A process remains on the CPU until its current CPU burst finishes or it blocks for I/O.

The average turnaround times are:

Process	Average TAT	Maximum TAT
1	363.33	590
2	525.00	600
3	14.50	16
4	123.00	163
5	189.00	225

P1 is a good baseline because it has no I/O. The long contiguous regions in its timeline are therefore a direct consequence of FIFO’s non-preemptive behaviour. P3 is the interesting counterexample: despite FIFO being the simplest policy, its turnaround is very small because the CPU bursts are only one unit long and the processes spend most of their time in I/O.

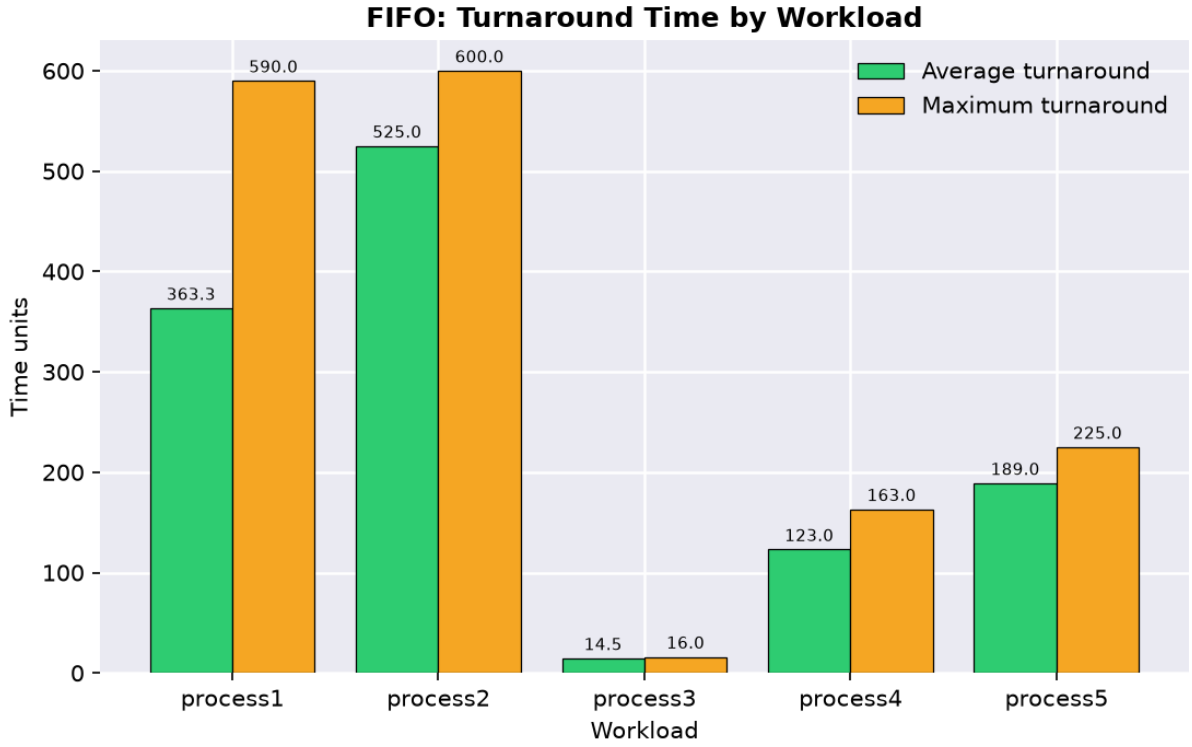


Figure 1: Average turnaround time under FIFO across the five workloads.

4.2 Looking at the actual schedules

The following pages show the actual simulated timelines for every workload. We use the same three configurations throughout this visual comparison: FIFO, RR with $q = 4$, and MLFQ with a 20-unit boost and quantum 2. The coloured blocks are not decorative: they show exactly where the scheduler is switching between processes.

Process 1

Process 1 is the cleanest CPU-bound case: two processes arrive at time 0 and a third arrives at time 10, with no I/O bursts. The long continuous blocks in the FIFO timeline therefore matter: once a process gets the CPU, there is no I/O event forcing it to leave. RR breaks those long blocks into slices, while MLFQ does the same initially and then changes priority as CPU time accumulates. This makes Process 1 useful for seeing scheduling overhead and queue policy without I/O complicating the picture.

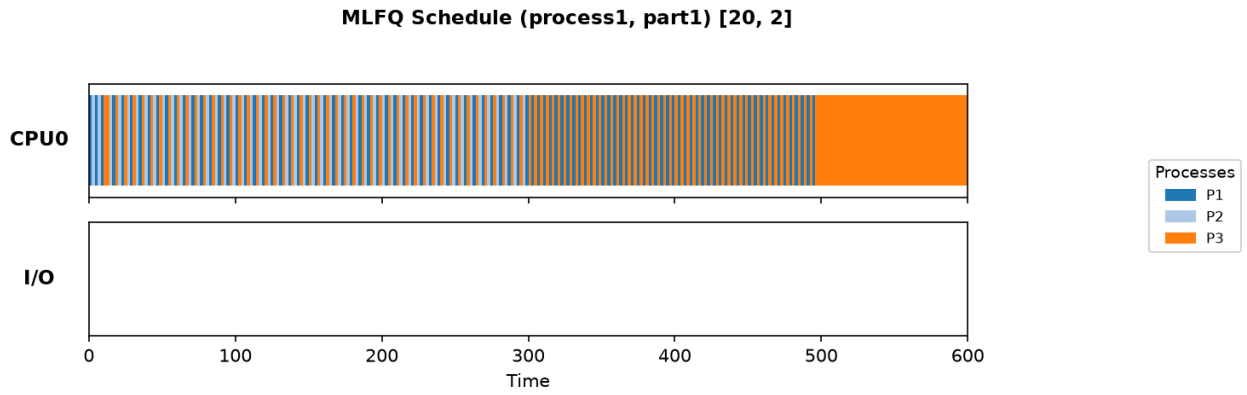
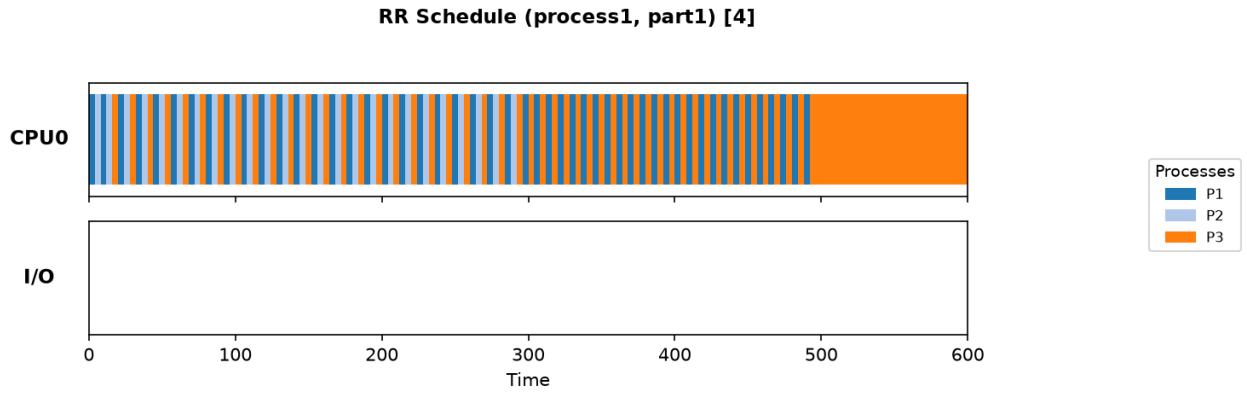
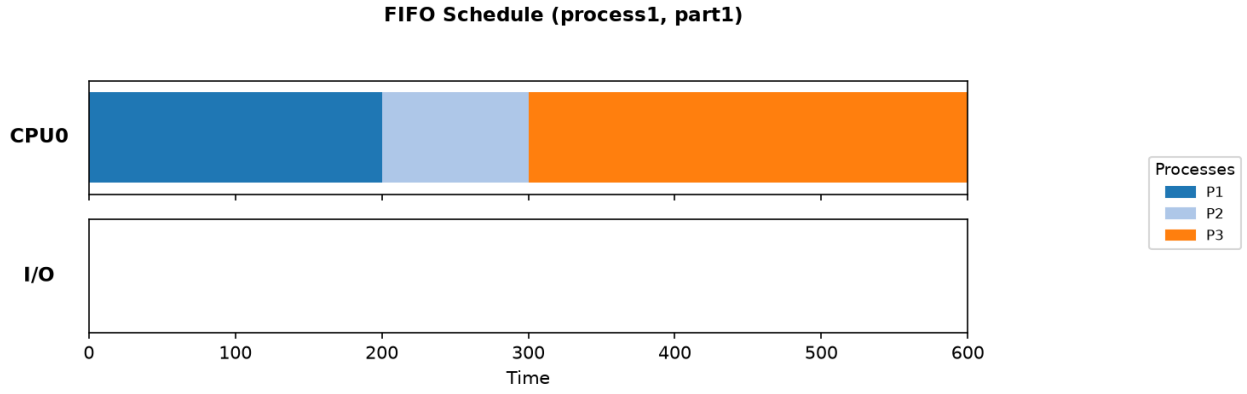


Figure 2: Scheduling timelines for Process 1, Part 1. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 2

Process 2 introduces repeated CPU–I/O cycles. The timelines now contain genuine gaps in a process’s execution caused by I/O, so the ready queue is constantly being repopulated. FIFO can still produce long CPU stretches, while RR produces regular interleaving. MLFQ’s behaviour is more interesting here because processes can leave for I/O and later return with their accumulated MLFQ state. The result is a good example of why a scheduler’s treatment of returning processes matters.

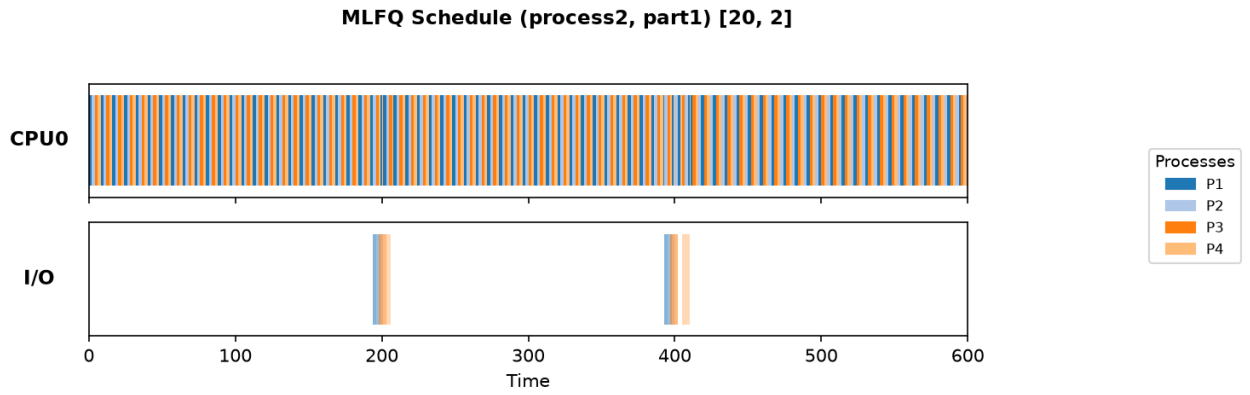
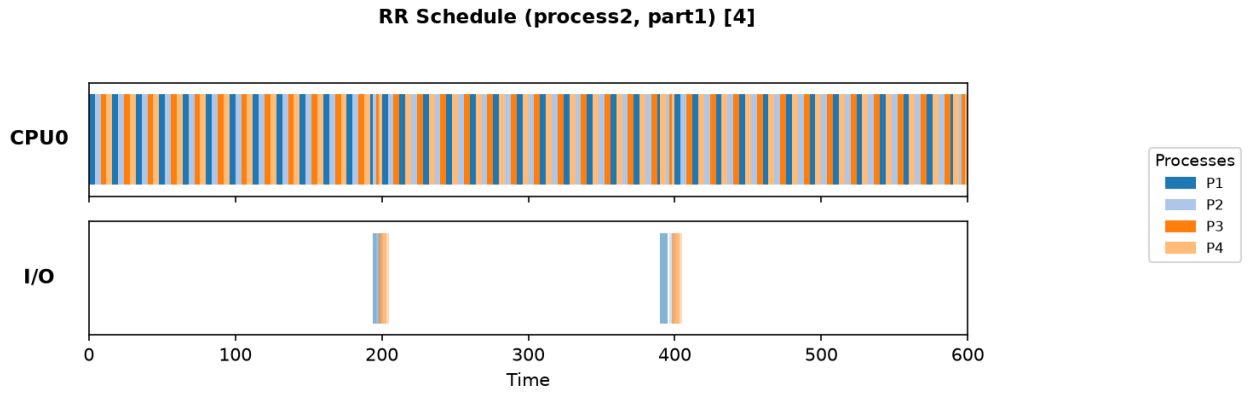
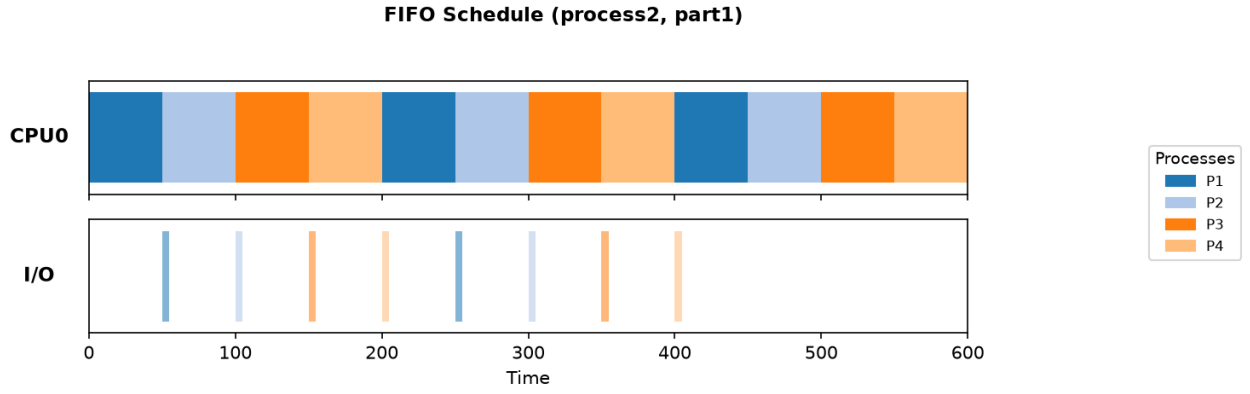
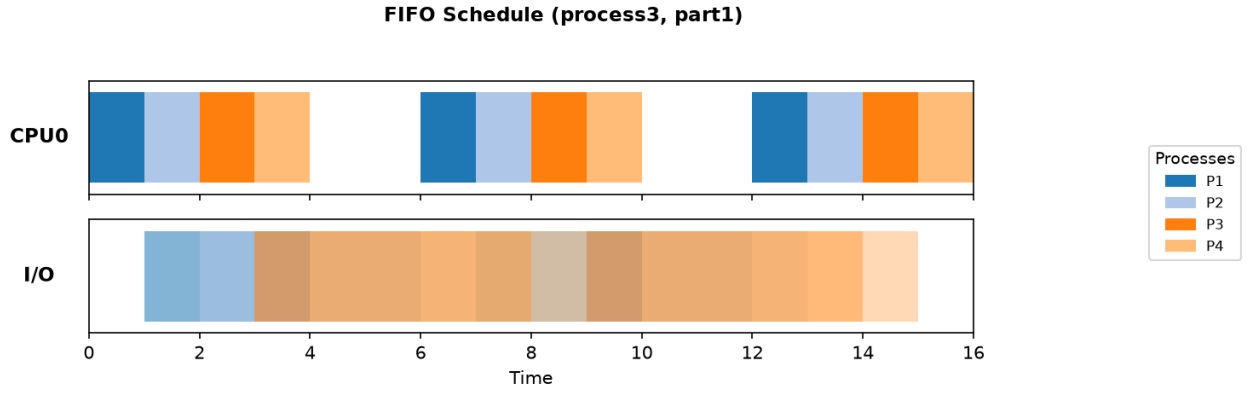


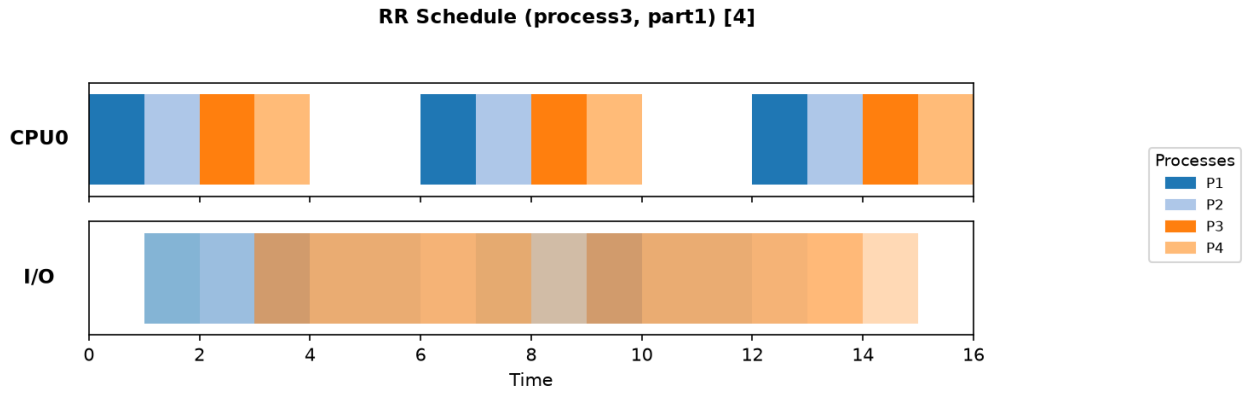
Figure 3: Scheduling timelines for Process 2, Part 1. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 3

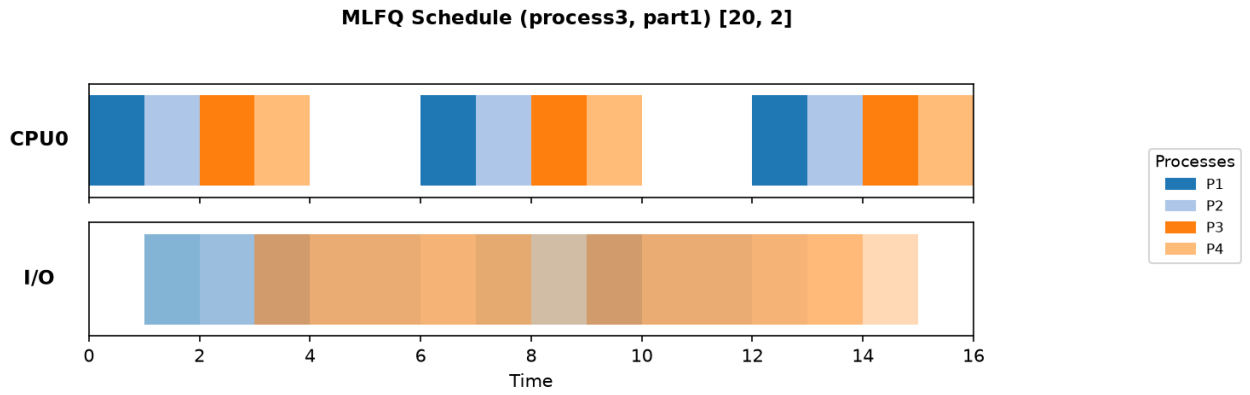
Process 3 is almost the opposite of Process 1. Its CPU bursts are only 1 time unit, separated by 5-unit I/O bursts. Consequently, the CPU scheduling policy has very little opportunity to improve turnaround: a process frequently leaves the CPU almost immediately because it has finished its CPU burst. This is also why all five RR quantum values produce exactly the same average turnaround of 14.50.



(a) FIFO



(b) Round Robin, $q = 4$

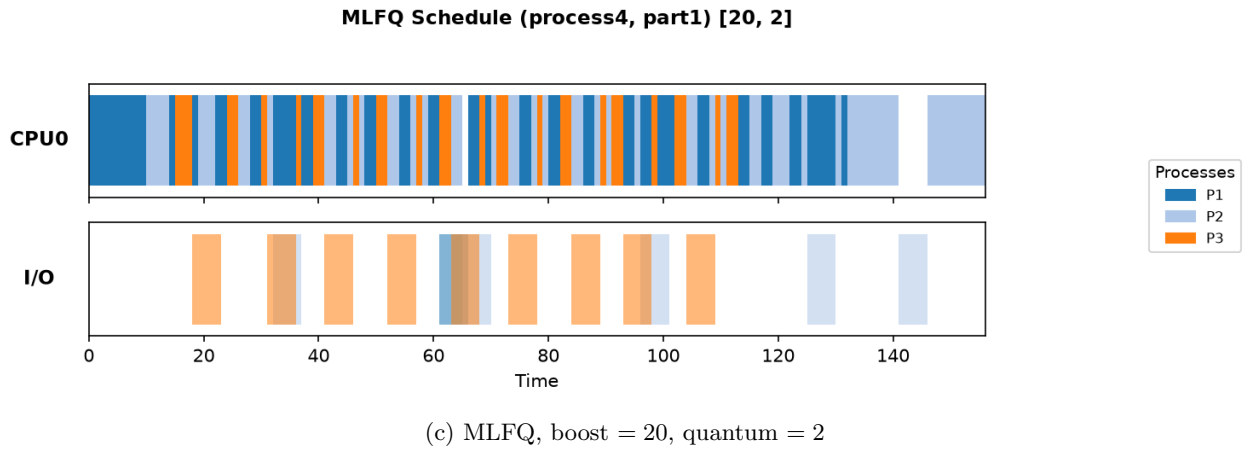
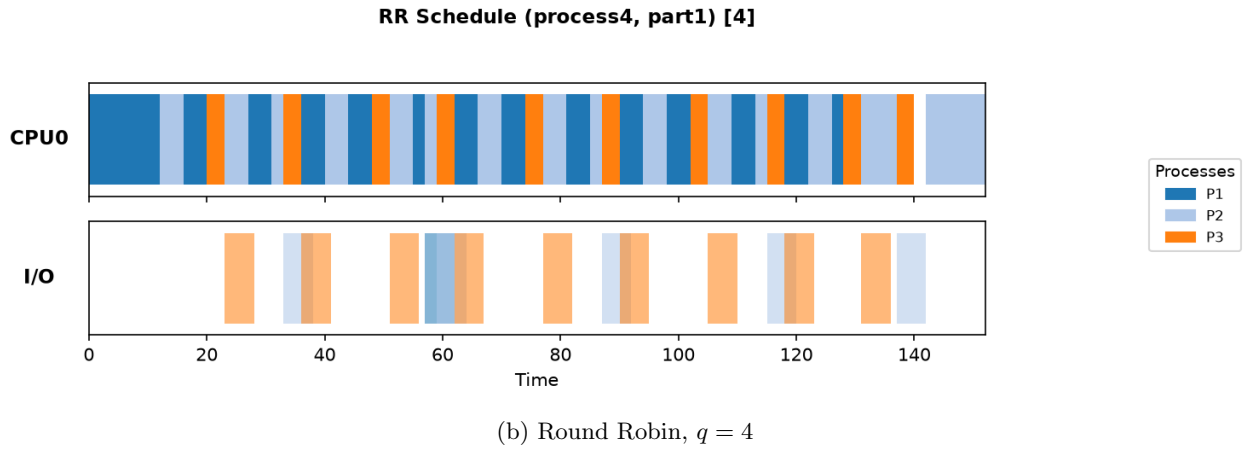
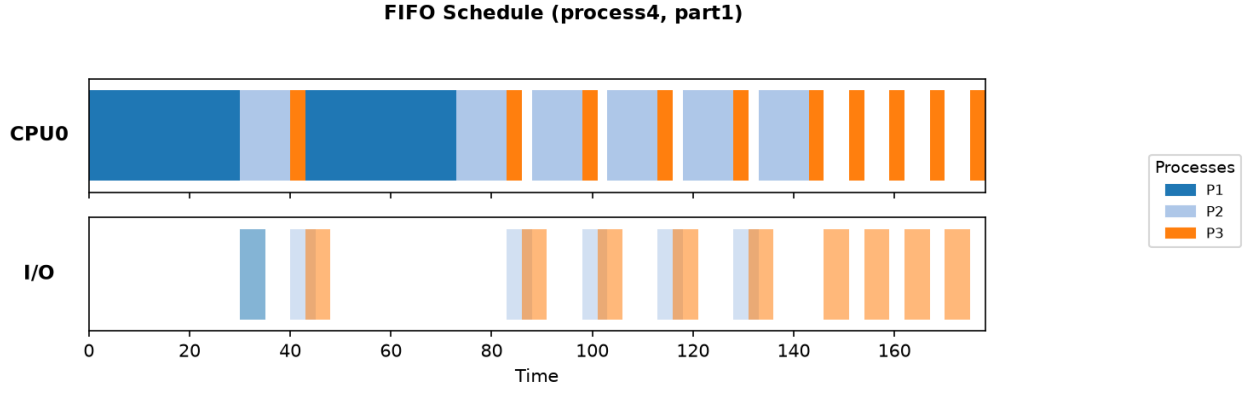


(c) MLFQ, boost = 20, quantum = 2

Figure 4: Scheduling timelines for Process 3, Part 1. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 4

Process 4 contains a much richer mixture of burst lengths and repeated I/O. This is where the differences between policies become visually useful. RR produces much finer-grained switching, while MLFQ gradually separates processes according to how much CPU time they have consumed. The 20-unit priority boost periodically collapses that separation and brings unfinished work back to the highest queue.



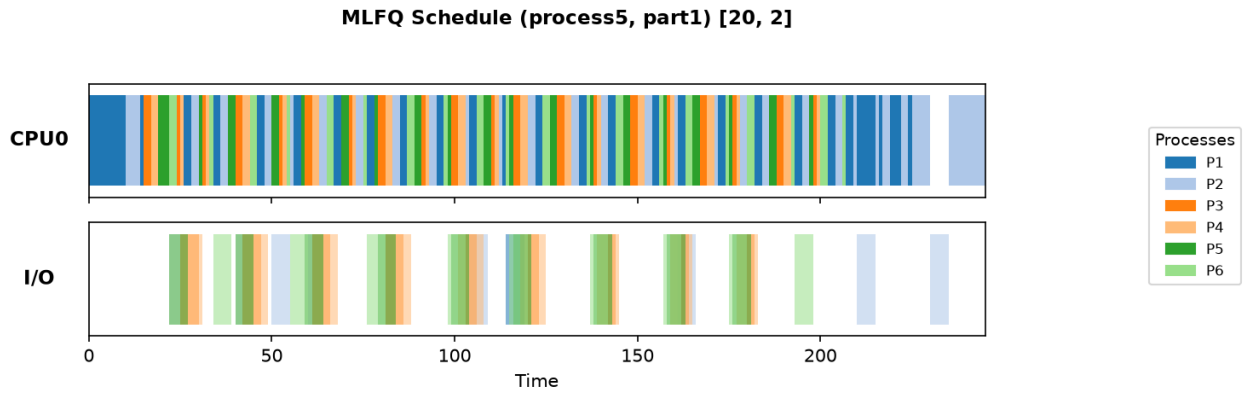
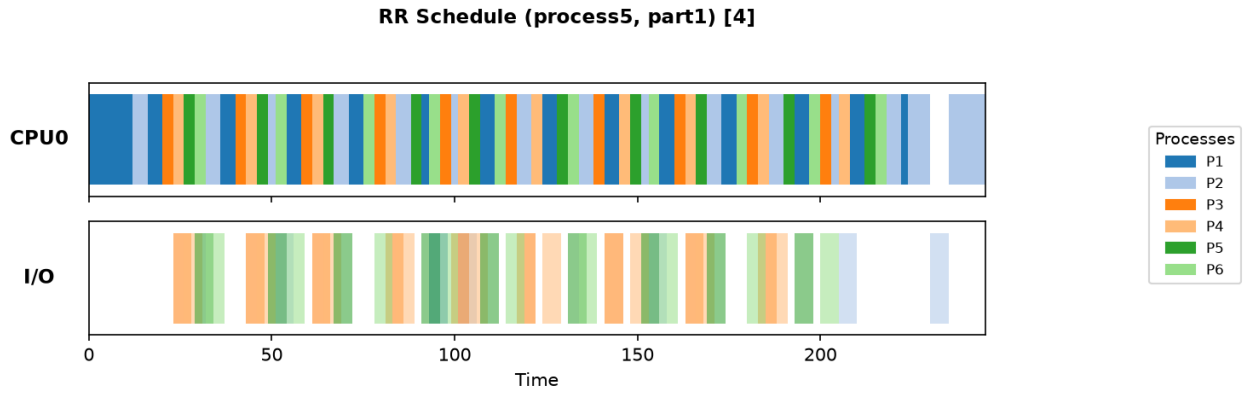
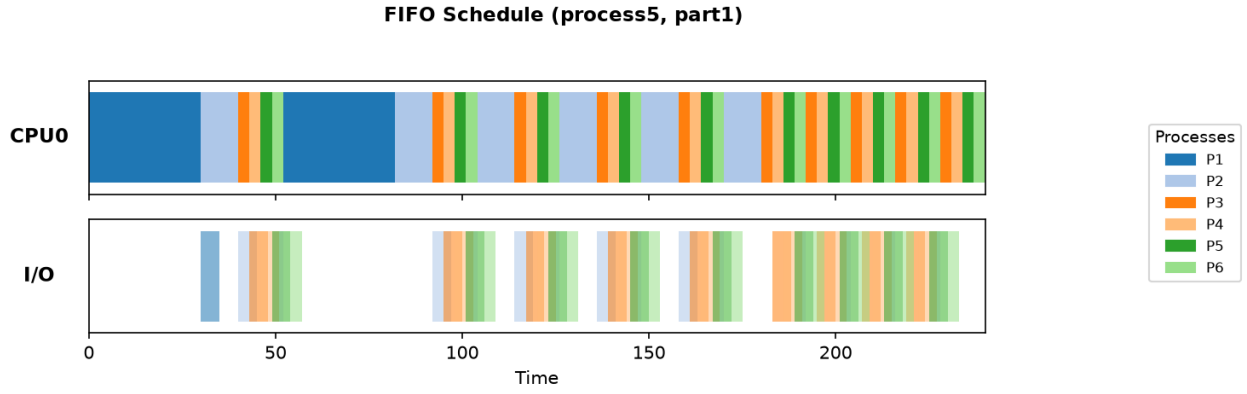


Figure 6: Scheduling timelines for Process 5, Part 1. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

What the one-CPU timelines tell us

There are three visual patterns worth noticing.

First, FIFO has relatively long contiguous blocks whenever the running process has a long CPU burst. There is no quantum forcing it off the CPU.

Second, RR creates much denser switching. A long CPU burst is repeatedly divided into turns, so the Gantt chart becomes visually “striped”. That does not automatically imply lower turnaround time; it only tells us that the policy is giving processes more frequent opportunities to run.

Third, MLFQ is not simply RR with a different name. Its execution pattern changes as a process

consumes CPU time and moves between priority levels. Periodic boosting then interrupts that long-term separation by returning unfinished work to Q_0 .

5. Round Robin: changing only the quantum

We evaluated five quantum values:

$$q \in \{1, 2, 4, 8, 16\}.$$

The purpose of this experiment is not to search for one universally optimal quantum, but to see how the quantum interacts with the burst structure of each workload.

P1

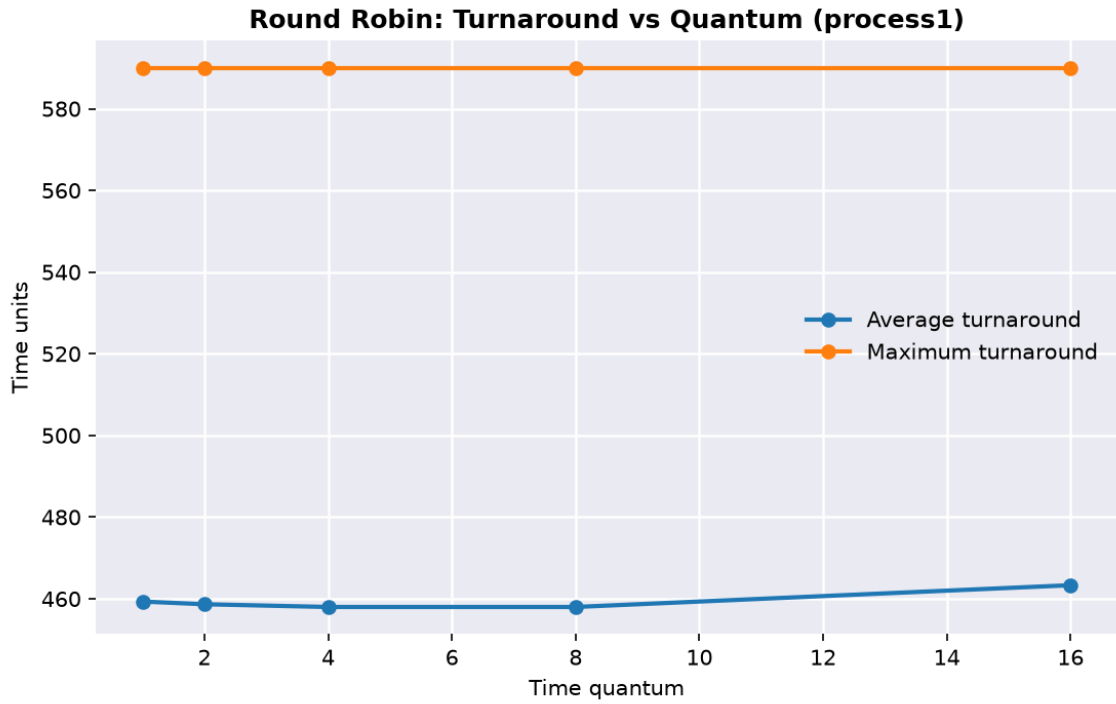


Figure 7: Average turnaround time of P1 as the RR quantum changes.

For P1, the average turnaround changes only slightly:

$$459.33, 458.67, 458.00, 458.00, 463.33$$

for $q = 1, 2, 4, 8, 16$.

Because P1 contains only three CPU-bound processes, changing the quantum changes their interleaving but does not introduce any I/O-driven arrivals back into the ready queue. This keeps the effect relatively small.

P2

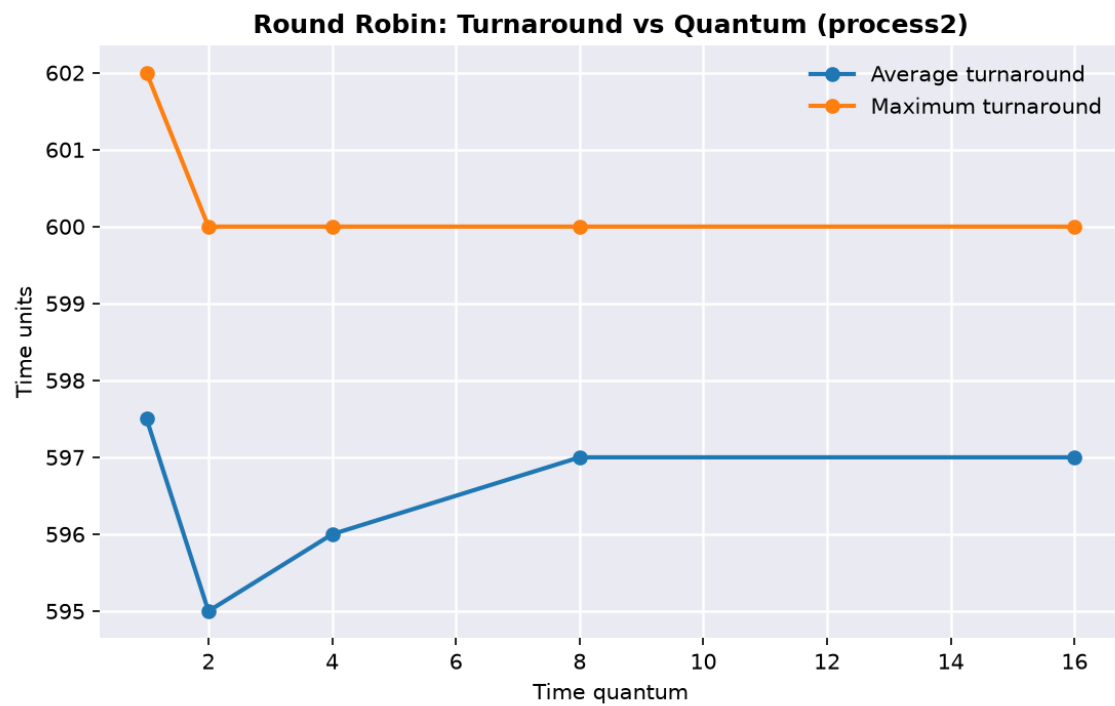


Figure 8: Average turnaround time of P2 as the RR quantum changes.

P2 gives:

597.50, 595.00, 596.00, 597.00, 597.00.

The values are again close. All processes have a similar CPU-I/O structure, so changing the quantum mostly changes the order in which otherwise similar work receives CPU time.

P3: the flat curve has a reason

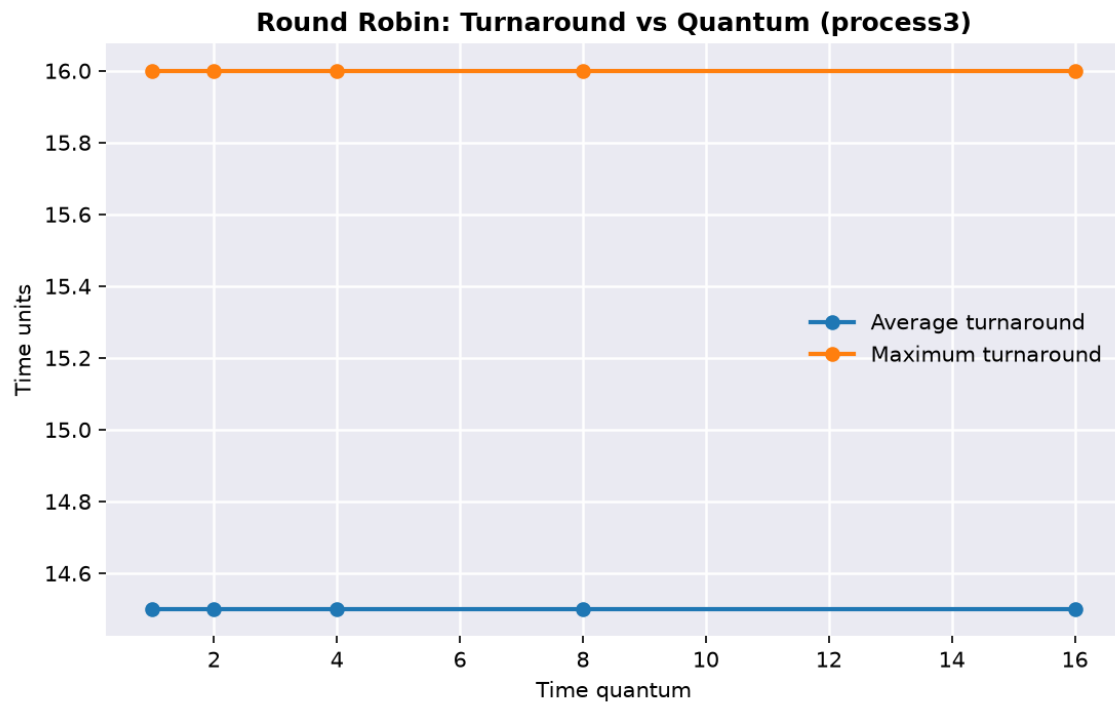


Figure 9: Average turnaround time of P3 as the RR quantum changes.

Every CPU burst in P3 is exactly one time unit. Hence even $q = 1$ is enough to complete a CPU burst in one turn. Increasing the quantum cannot make the CPU burst any more complete.

Consequently:

$$T_{\text{avg}} = 14.50$$

for all five quantum values.

This is a useful sanity check for the simulator: the quantum should matter only when it can actually change how CPU bursts are divided.

P4

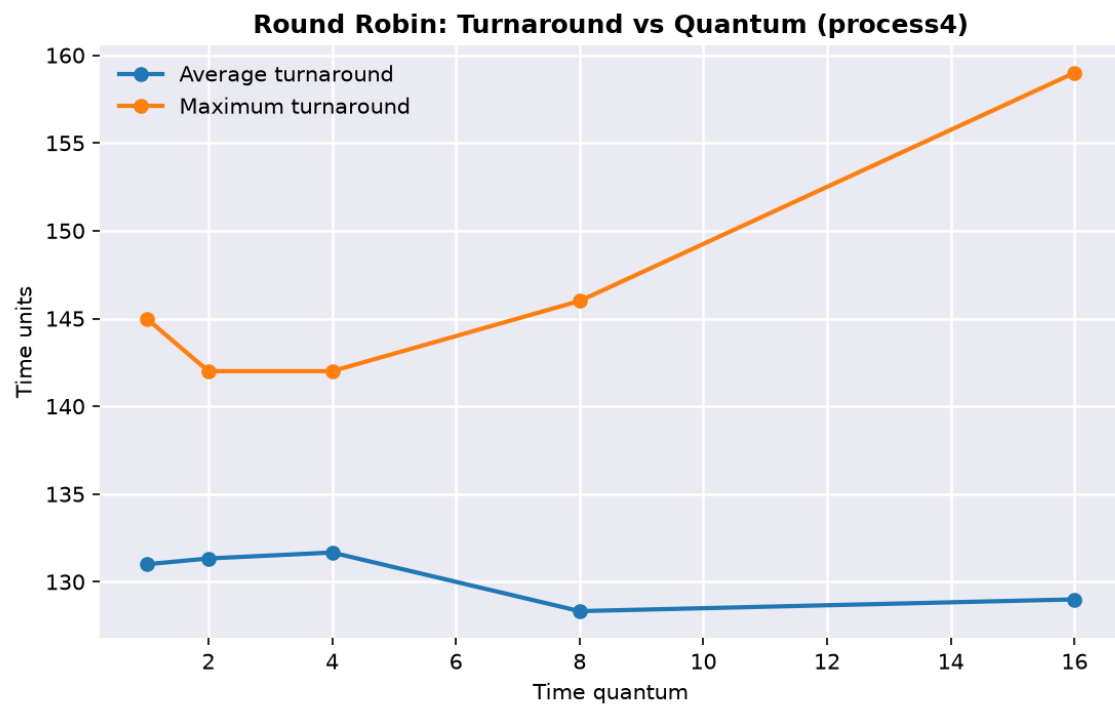


Figure 10: Average turnaround time of P4 as the RR quantum changes.

P4 produces:

131.00, 131.33, 131.67, 128.33, 129.00.

The change is still moderate, but the curve is no longer flat because P4 has a mixture of burst lengths and repeated I/O. A quantum of 8 changes the execution pattern enough to reduce the average turnaround relative to $q = 1, 2, 4$ in this workload.

P5

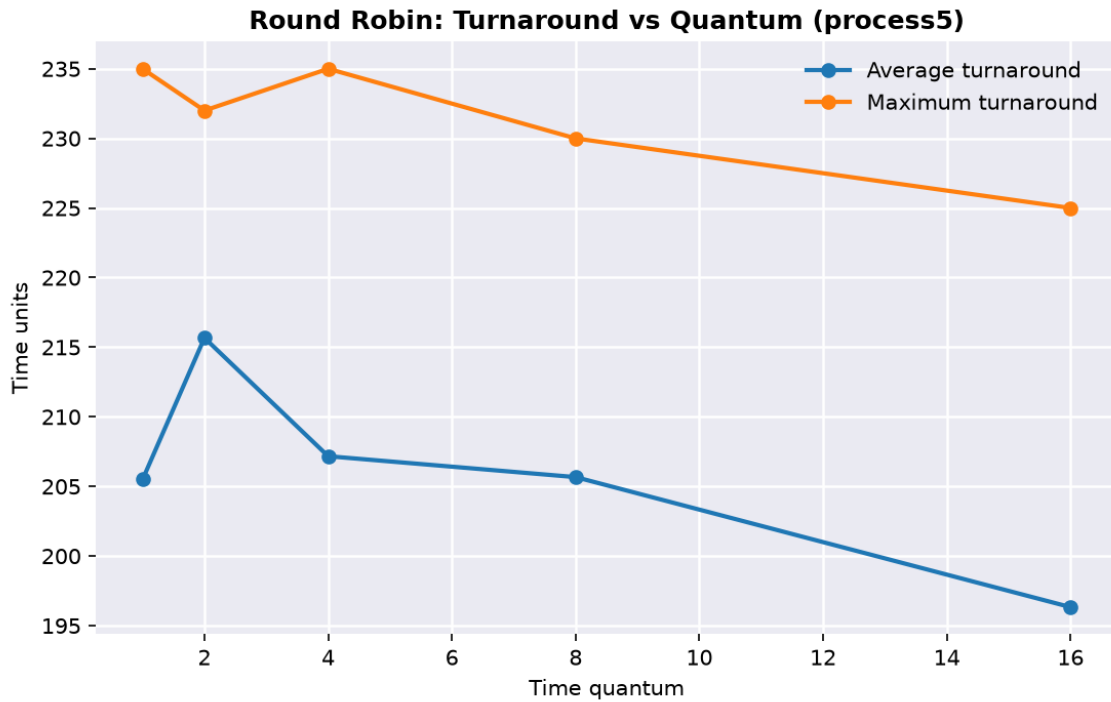


Figure 11: Average turnaround time of P5 as the RR quantum changes.

P5 is the most informative RR experiment:

205.50, 215.67, 207.17, 205.67, 196.33.

The increase at $q = 2$ followed by the decrease at larger quanta is a reminder that RR quantum selection is a workload-dependent scheduling decision. A larger quantum can reduce the number of forced switches through longer CPU bursts, while a very small quantum can repeatedly interrupt them. The exact turnaround consequence depends on which processes are waiting and which processes return from I/O at each point in the simulation.

6. MLFQ: priority boost experiment

The required comparison is between no boost and a priority boost every 20 simulated time units.

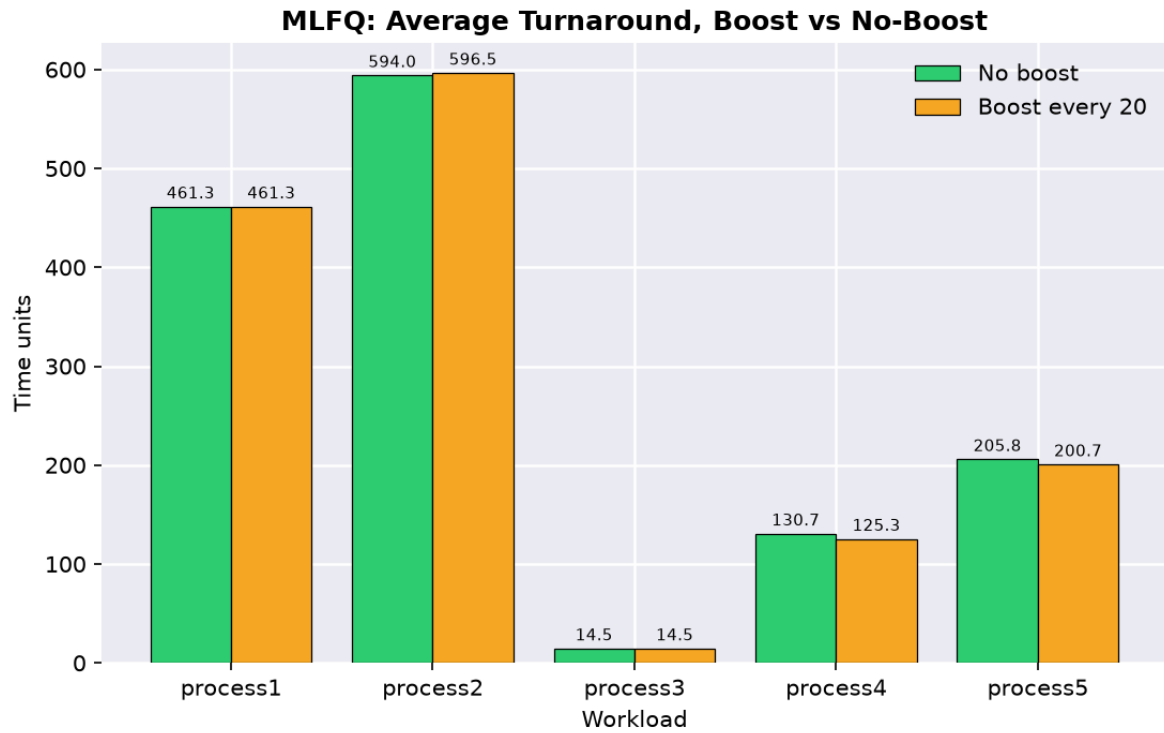


Figure 12: MLFQ average turnaround: no boost versus boost every 20 time units.

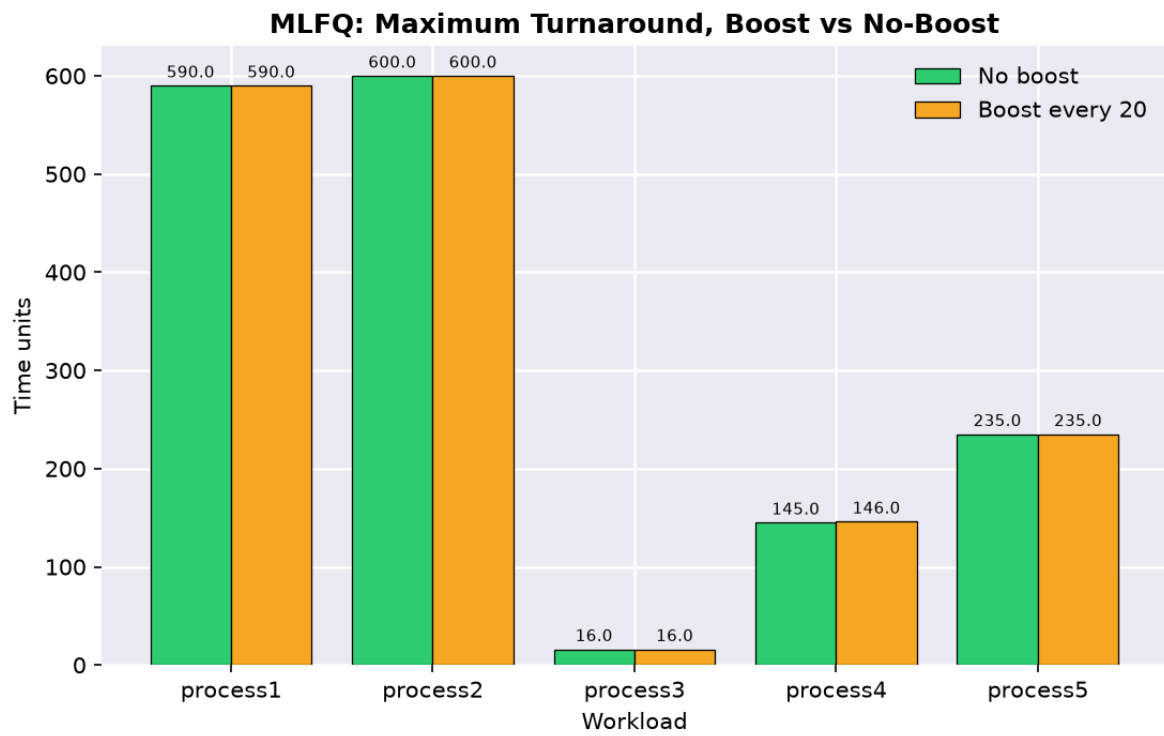


Figure 13: MLFQ maximum turnaround: no boost versus boost every 20 time units.

The average TAT values are:

Process	No boost	Boost 20	Change
1	461.33	461.33	0.00
2	594.00	596.50	+2.50
3	14.50	14.50	0.00
4	130.67	125.33	-5.34
5	205.83	200.67	-5.16

The most important point is that the boost does not have the same effect on every workload.

For P1, there is no change. With only three CPU-bound processes, the periodic reset does not create a different completion pattern.

For P3, there is again no change. Its one-unit CPU bursts and five-unit I/O bursts leave very little room for a priority policy to alter the amount of useful CPU work completed before a process blocks again.

P4 and P5 respond differently. Both contain many short CPU bursts and repeated returns from I/O, so the priority boost changes which processes get access to Q_0 and when. Their average turnaround therefore decreases.

P2 is particularly useful because its average TAT actually increases slightly. This shows why it would be misleading to describe a periodic boost simply as an “improvement”. It changes the scheduling order; whether that order reduces a particular metric depends on the workload.

The maximum turnaround results reinforce the same point. P4’s average improves from 130.67 to 125.33, but its maximum changes from 145 to 146. Average and worst-case behaviour therefore need to be considered separately.

A closer look at the MLFQ queues

The queue-level plots make the previous discussion much easier to interpret than the aggregate Gantt chart alone.

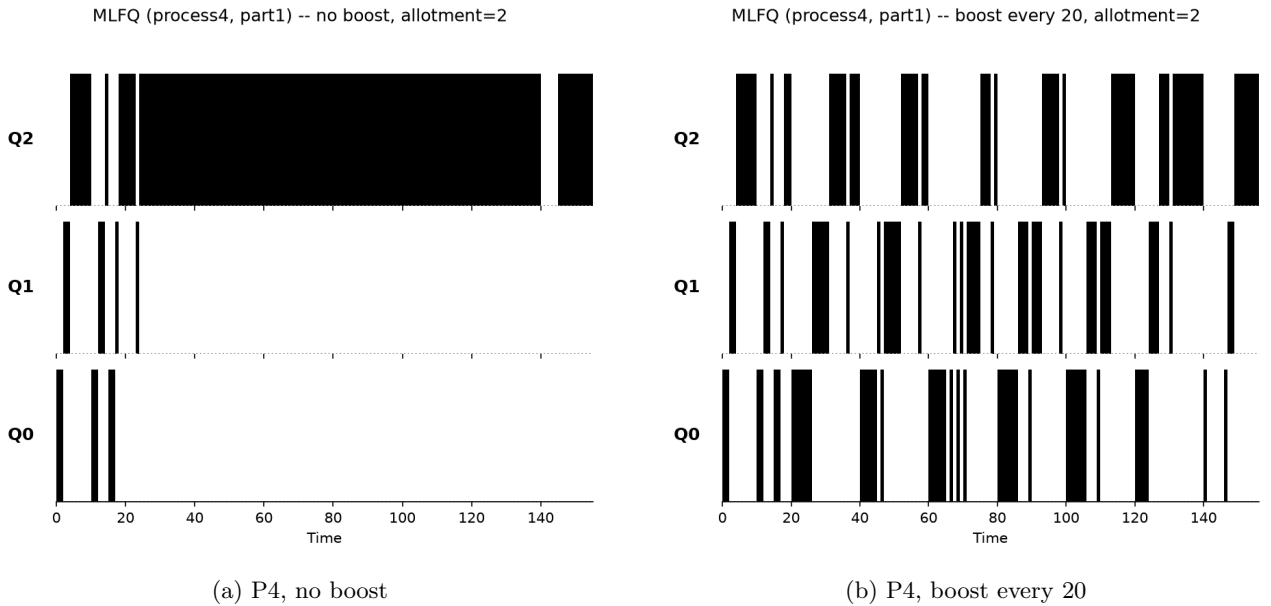


Figure 14: Queue-level view of MLFQ for P4. The rows correspond to Q_2 , Q_1 and Q_0 .

Without boosting, work can remain in lower-priority levels after repeated CPU use. The boost periodically collapses this separation and returns unfinished processes to Q_0 . For a workload such as

P4, which repeatedly creates short bursts and I/O returns, that changes the ordering often enough to affect turnaround.

P5 shows the same mechanism with even more activity:

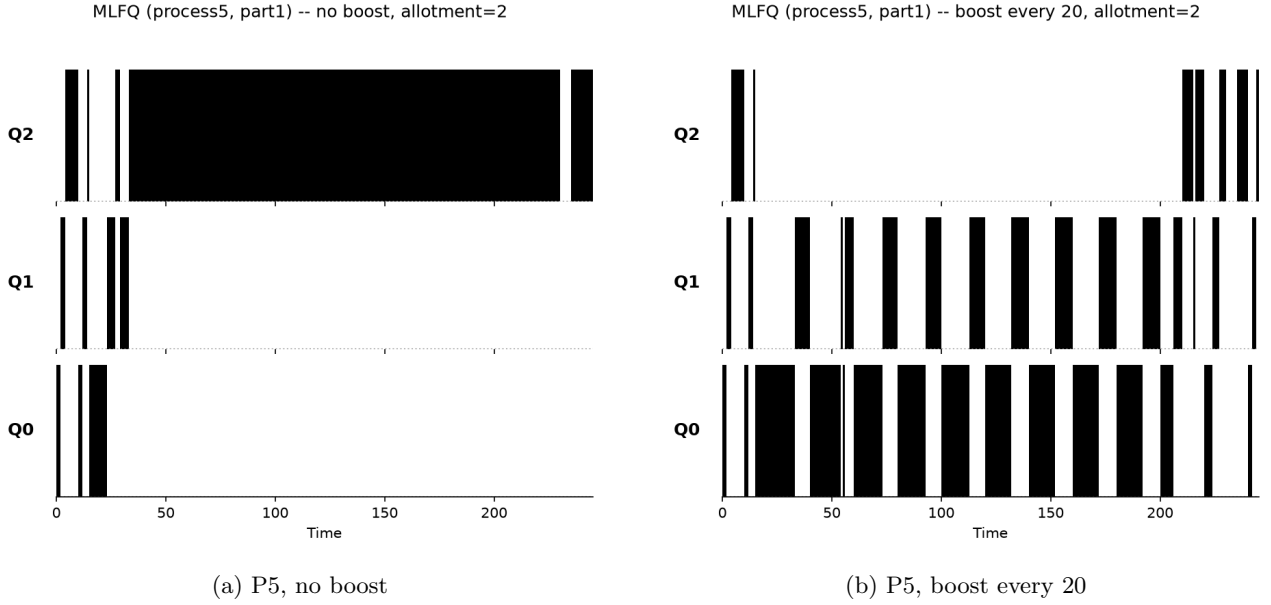


Figure 15: Queue-level view of MLFQ for P5.

7. MLFQ allotment: an additional experiment

We also varied the cumulative allotment at an MLFQ level:

$$A \in \{2, 6, 10, 20\}.$$

This is an additional experiment beyond the required boost comparison. It is useful because it separates two ideas that are easy to conflate: the quantum controls the length of a single turn, while the allotment controls how much cumulative CPU time a process can receive at that level before demotion.

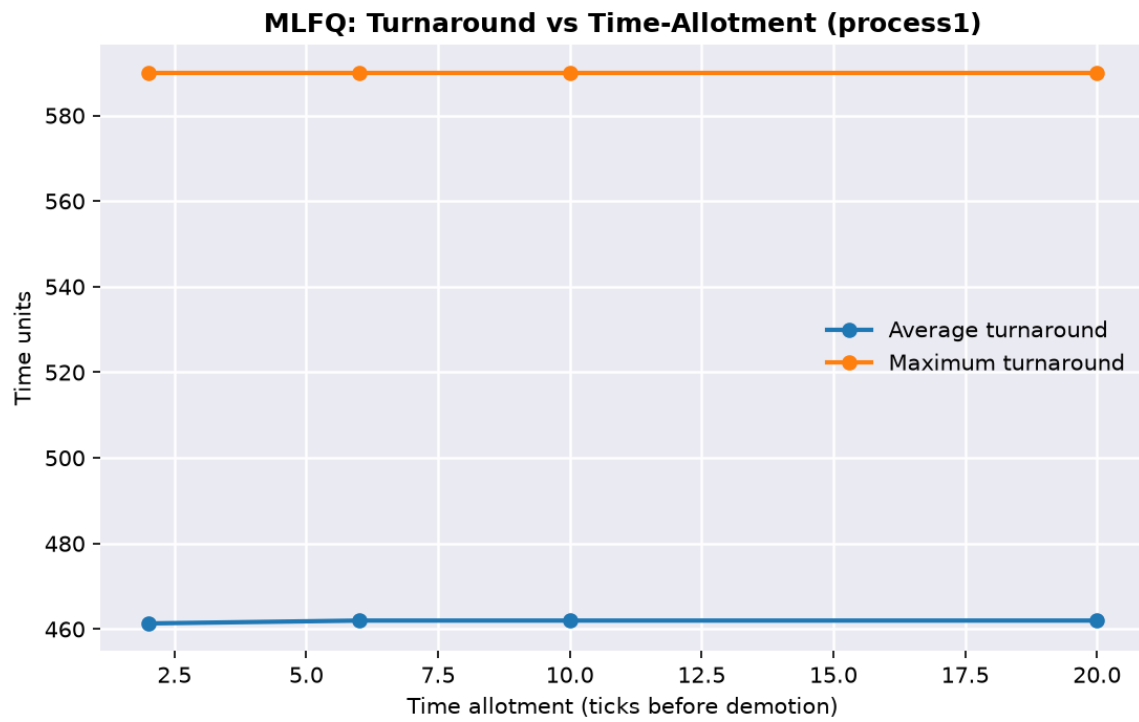


Figure 16: MLFQ average turnaround for P1 as allotment changes.

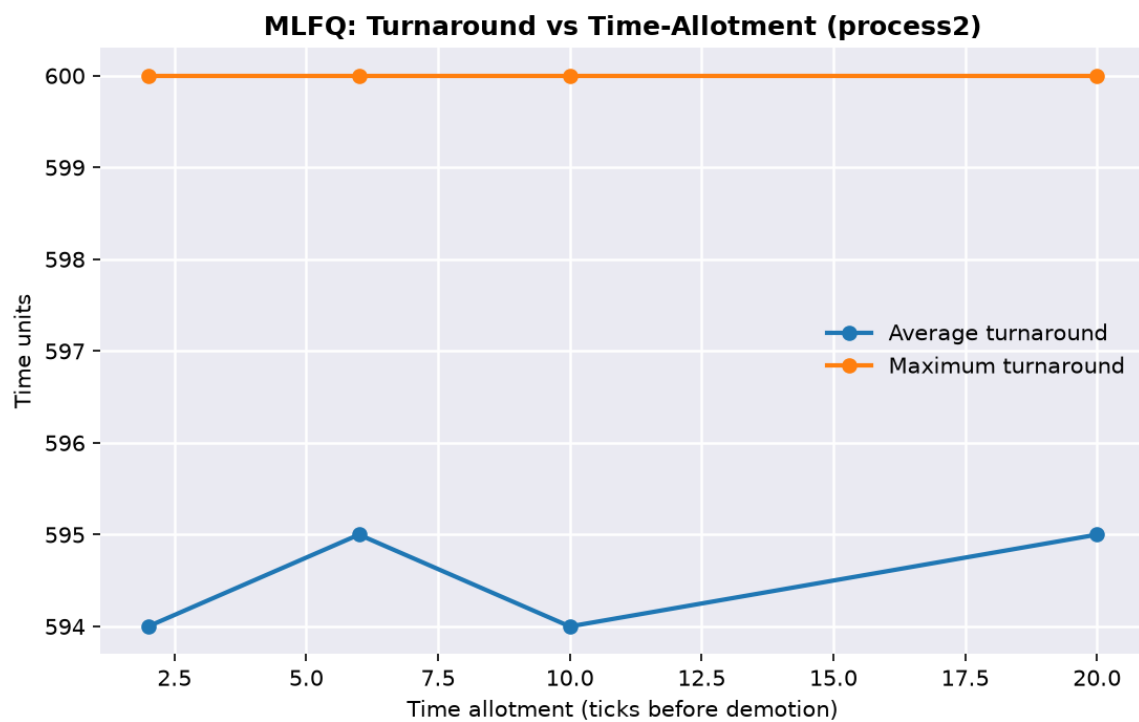


Figure 17: MLFQ average turnaround for P2 as allotment changes.

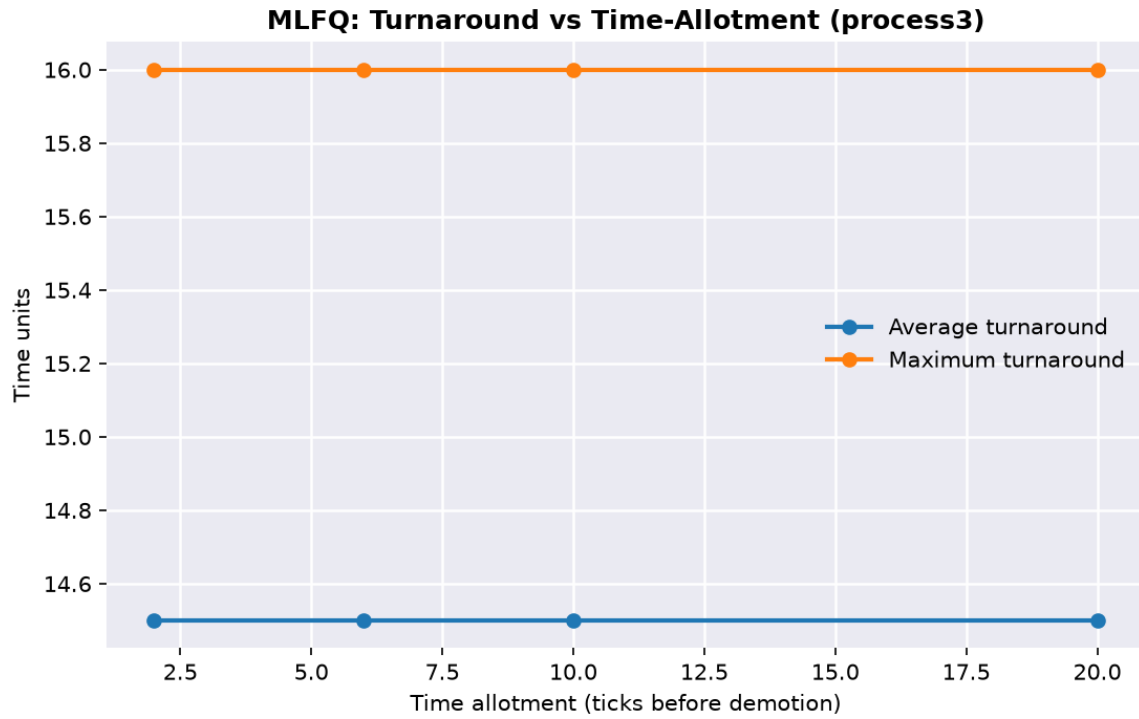


Figure 18: MLFQ average turnaround for P3 as allotment changes.

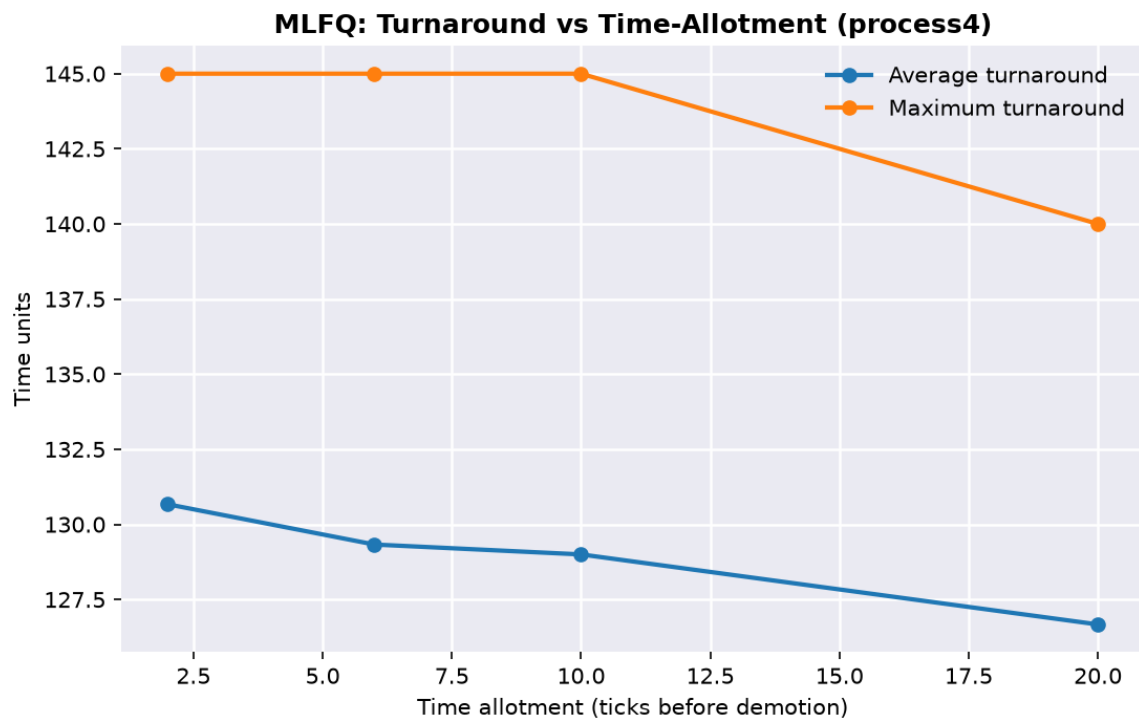


Figure 19: MLFQ average turnaround for P4 as allotment changes.

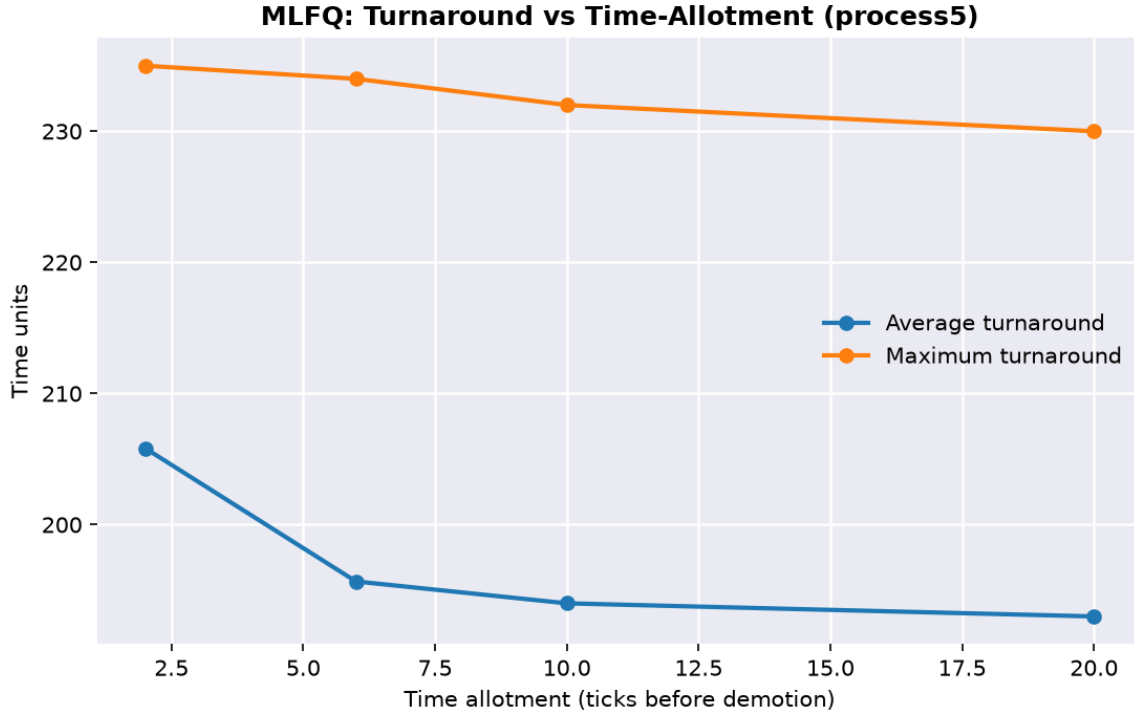


Figure 20: MLFQ average turnaround for P5 as allotment changes.

P3 is again essentially unchanged because its CPU bursts are so short. P5 shows the largest movement:

$$205.83 \rightarrow 195.67 \rightarrow 194.00 \rightarrow 193.00.$$

Here, allowing more cumulative CPU time before demotion changes how long the more CPU-intensive work remains at higher levels.

The important implementation insight is that the allotment counter must persist across an I/O block. Otherwise a process could repeatedly reset its demotion progress simply by performing I/O, which would make the intended cumulative-allotment behaviour disappear.

8. Part II – Two processors

The second part repeats the scheduling study with two processors. We keep the same policies and compare the average turnaround time obtained with one CPU and two CPUs.

Process	FIFO		RR ($q = 4$)		MLFQ (boost 20)	
	1 CPU	2 CPU	1 CPU	2 CPU	1 CPU	2 CPU
1	363.33	230.00	458.00	244.67	461.33	245.33
2	525.00	275.00	596.00	305.00	596.50	305.50
3	14.50	13.50	14.50	13.50	14.50	13.50
4	123.00	81.33	131.67	80.33	125.33	78.00
5	189.00	104.17	207.17	106.00	200.67	106.17

The reduction for P1 and P2 is substantial because there is enough independent CPU work to occupy both processors.

P3 is the important contrast. Its average turnaround changes only from 14.50 to 13.50. The second CPU does not shorten the five-unit I/O intervals. Once the processes are blocked for I/O, another processor cannot make those I/O events complete sooner.

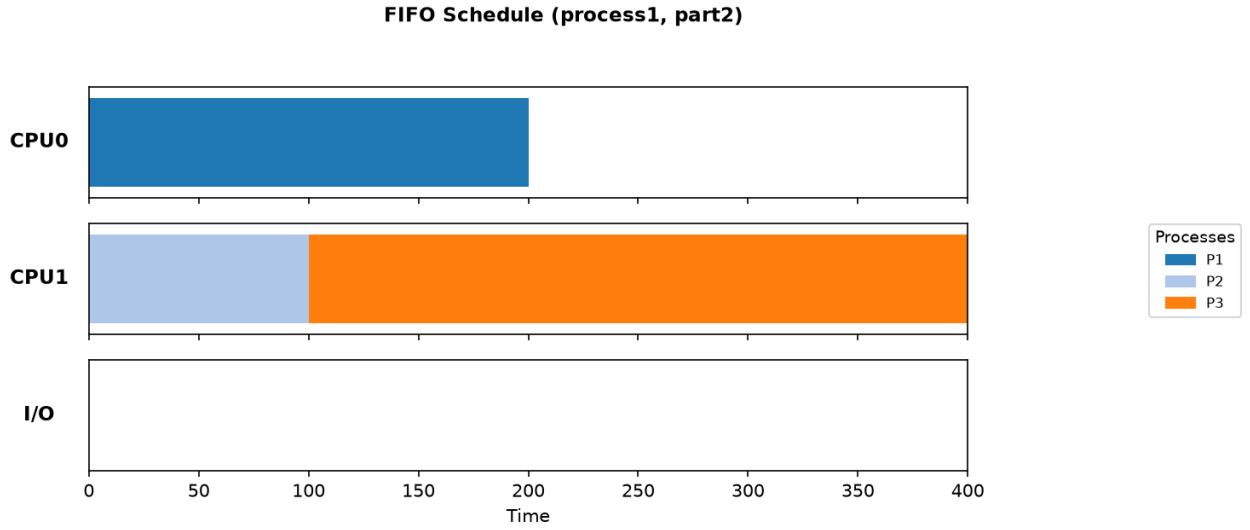
This is a useful example of why processor count and workload parallelism have to be considered together.

Two-CPU schedules

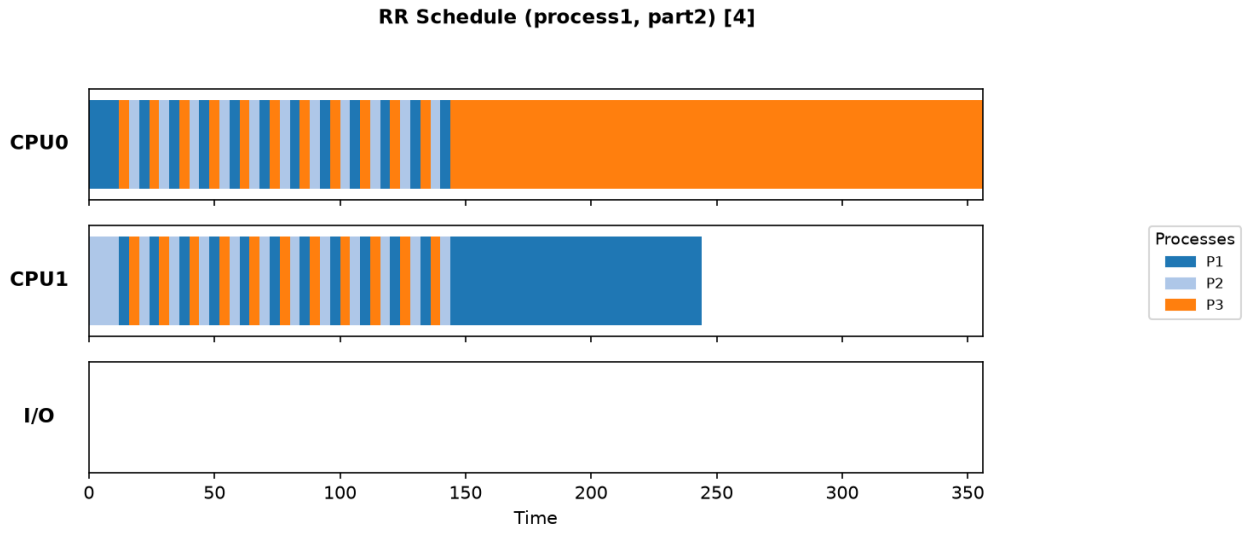
The following timelines show the same three representative policies for all five workloads with two CPUs.

Process 1 on two CPUs

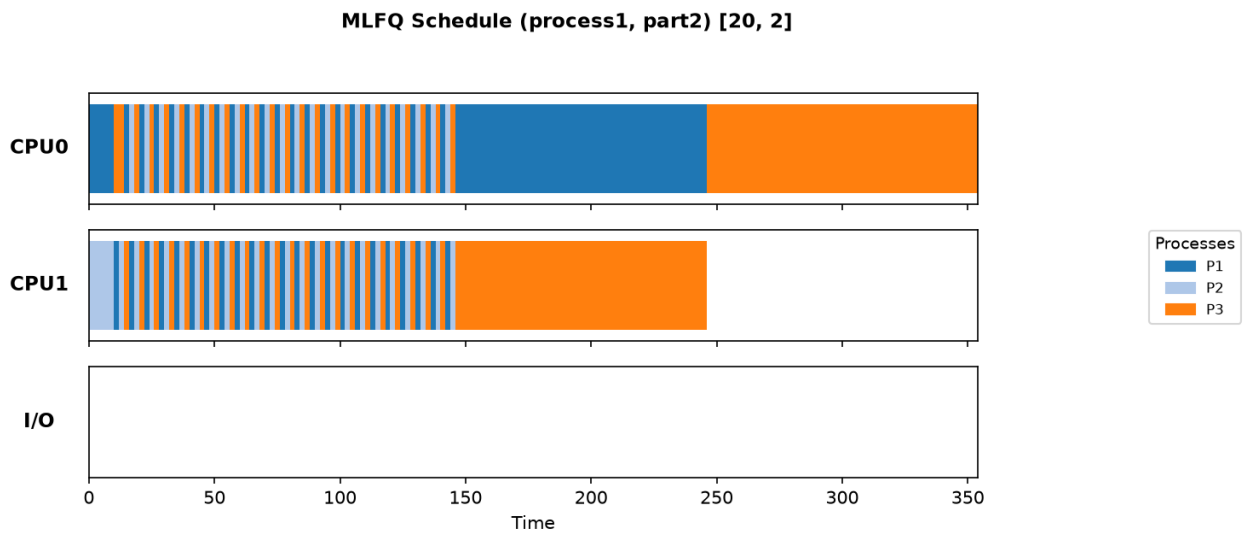
The main visual difference from Part I is that two independent CPU executions can appear at the same simulated time. When several processes are ready, the second processor removes part of the queueing delay that existed with a single CPU.



(a) FIFO



(b) Round Robin, $q = 4$

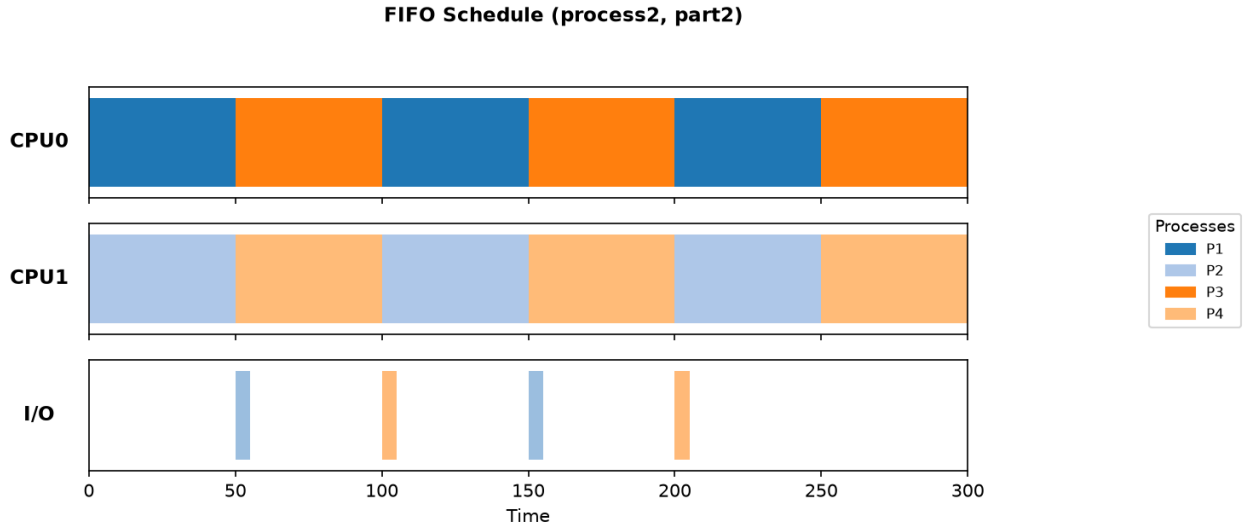


(c) MLFQ, boost = 20, quantum = 2

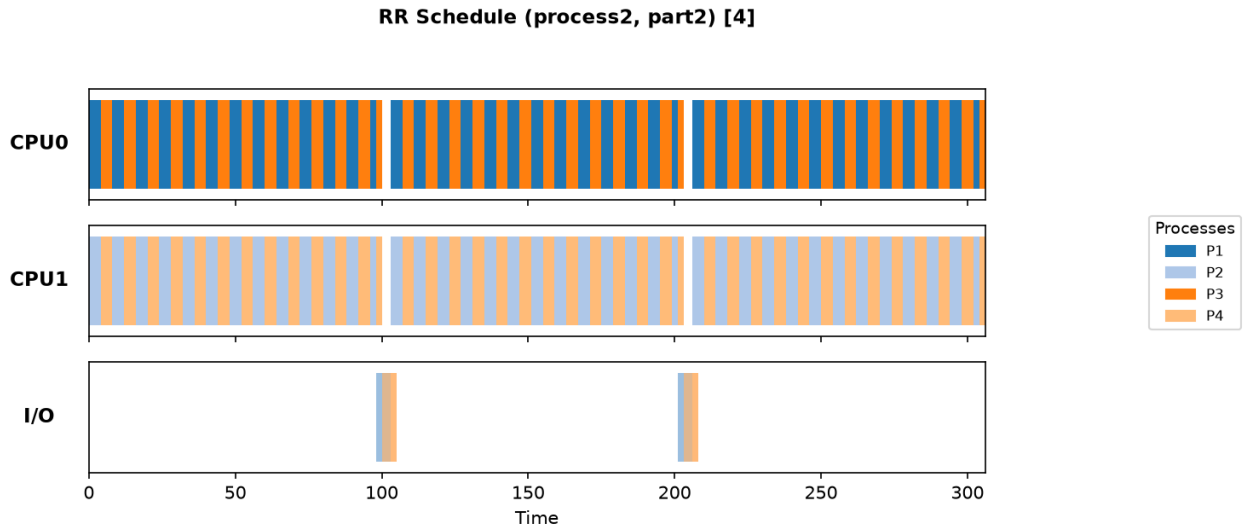
Figure 21: Scheduling timelines for Process 1, Part 2. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 2 on two CPUs

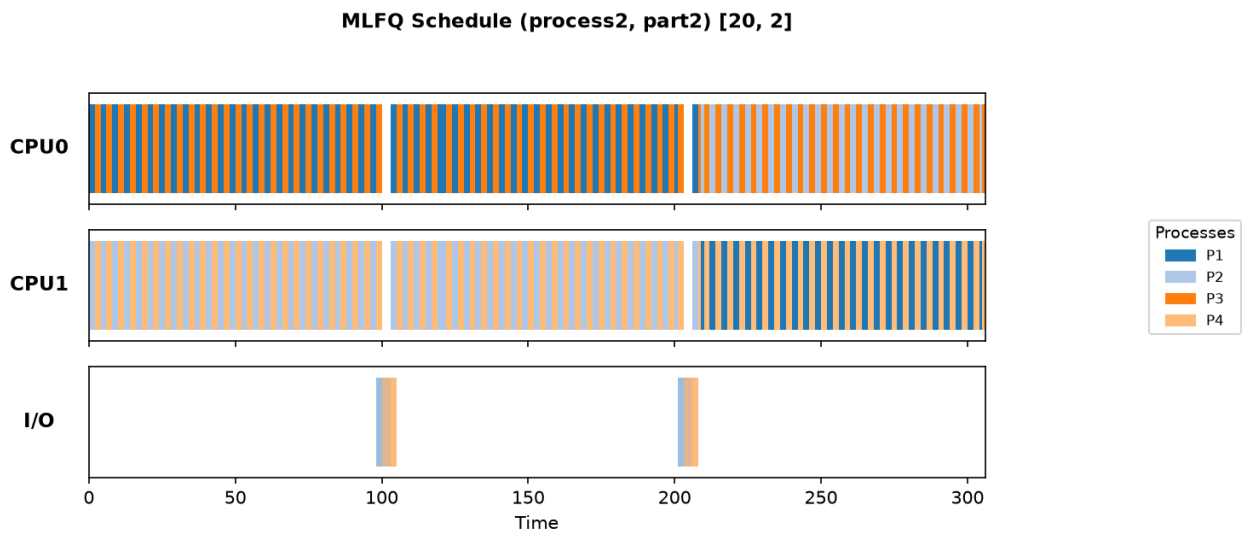
The main visual difference from Part I is that two independent CPU executions can appear at the same simulated time. When several processes are ready, the second processor removes part of the queueing delay that existed with a single CPU.



(a) FIFO



(b) Round Robin, $q = 4$

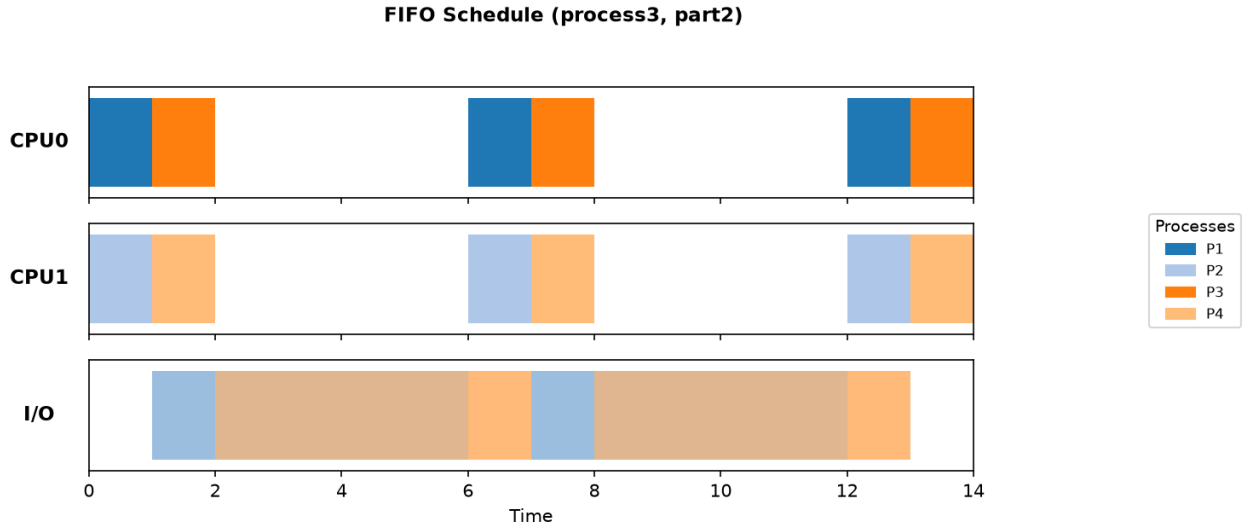


(c) MLFQ, boost = 20, quantum = 2

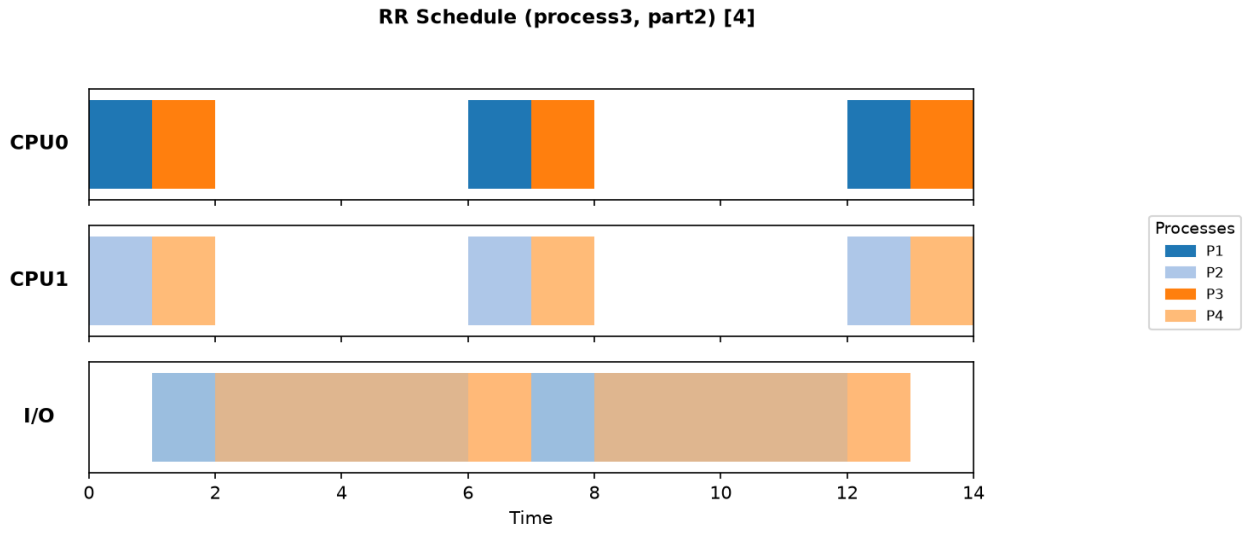
Figure 22: Scheduling timelines for Process 2, Part 2. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 3 on two CPUs

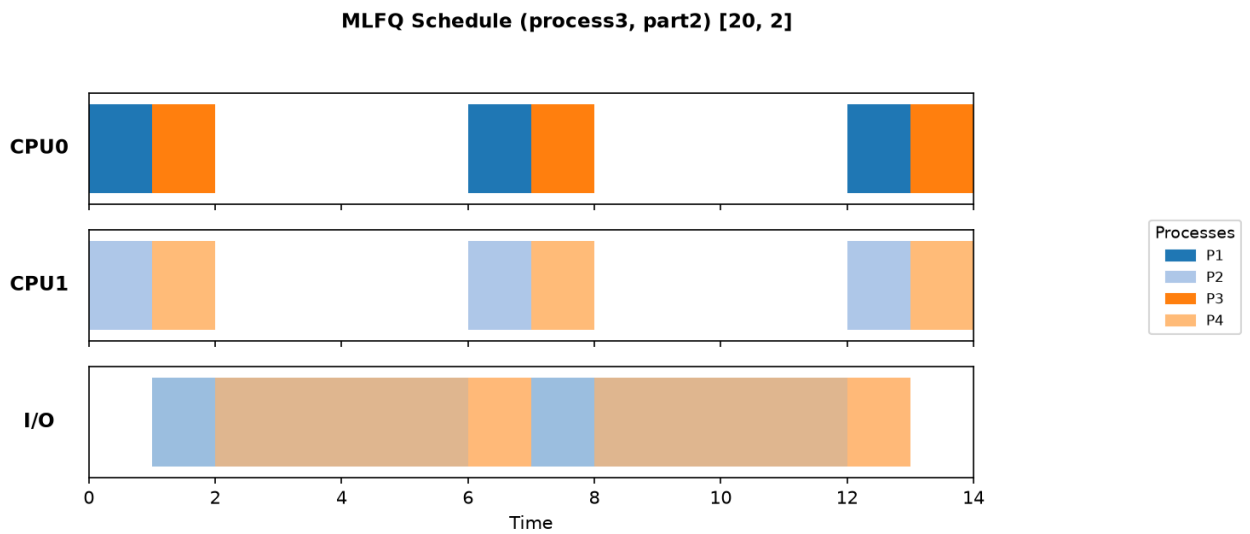
The main visual difference from Part I is that two independent CPU executions can appear at the same simulated time. When several processes are ready, the second processor removes part of the queueing delay that existed with a single CPU.



(a) FIFO



(b) Round Robin, $q = 4$

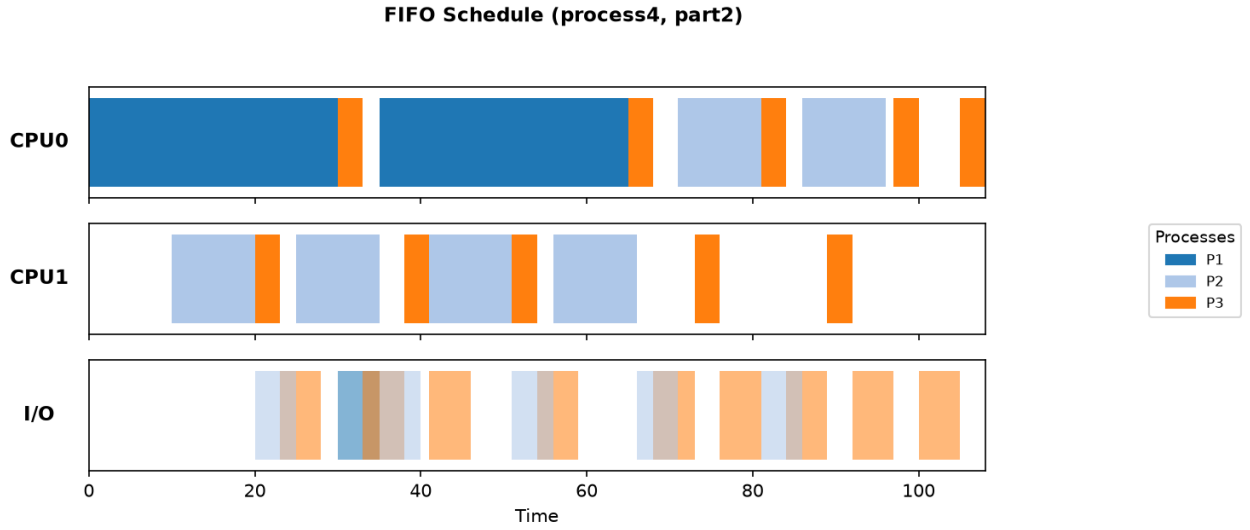


(c) MLFQ, boost = 20, quantum = 2

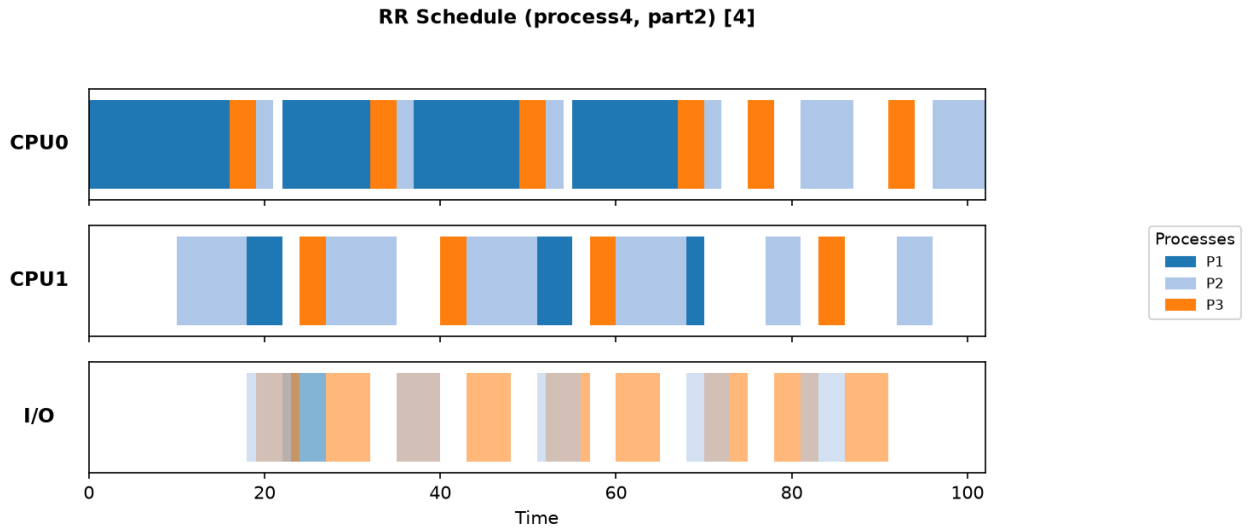
Figure 23: Scheduling timelines for Process 3, Part 2. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

Process 4 on two CPUs

The main visual difference from Part I is that two independent CPU executions can appear at the same simulated time. When several processes are ready, the second processor removes part of the queueing delay that existed with a single CPU.

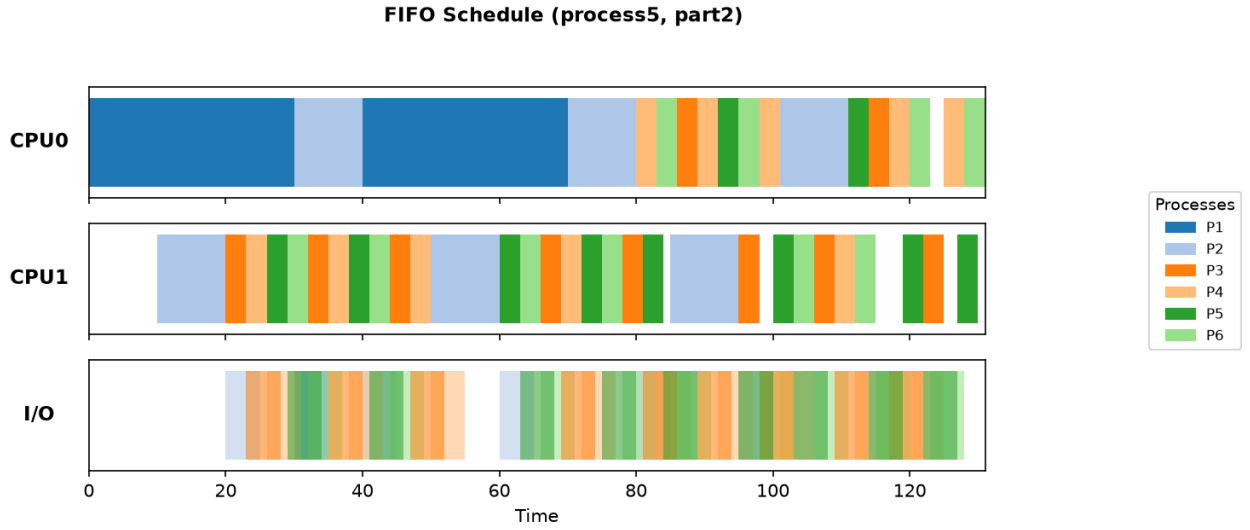


(a) FIFO

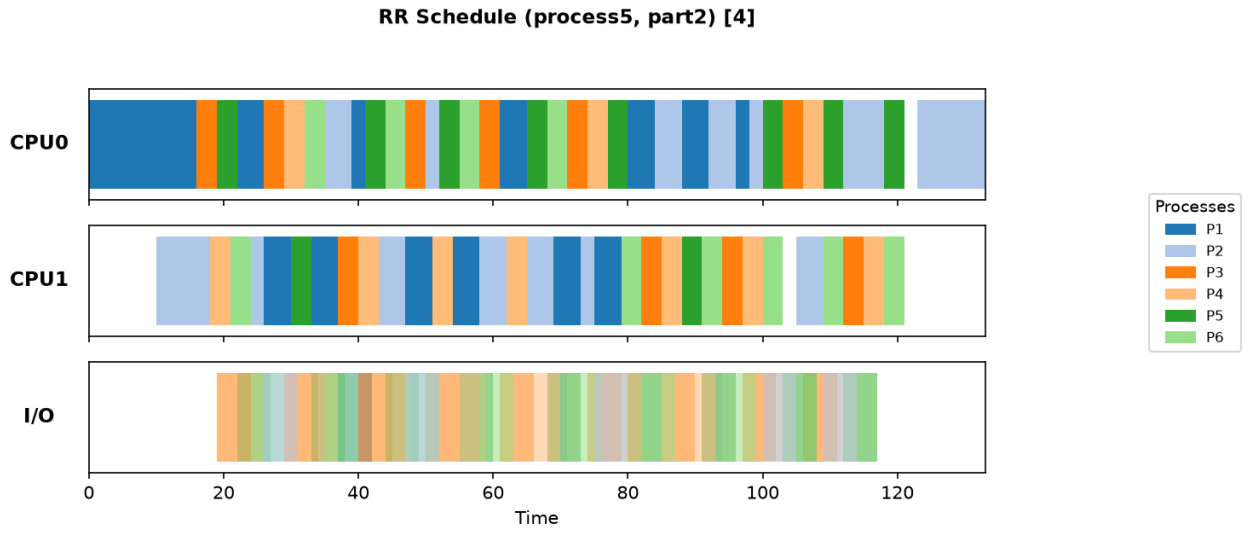


Process 5 on two CPUs

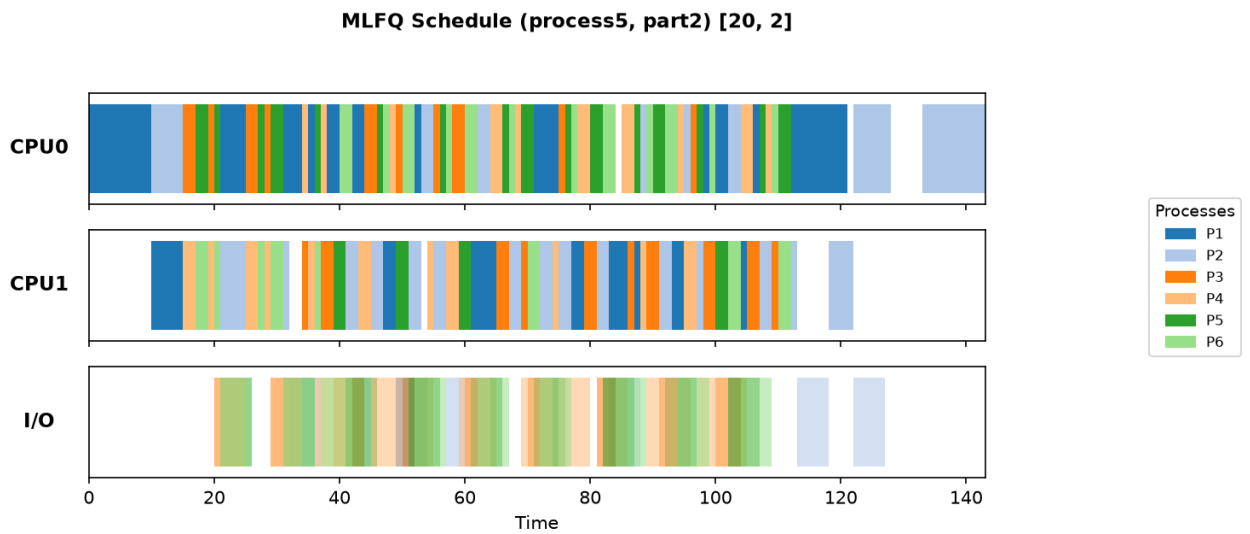
The main visual difference from Part I is that two independent CPU executions can appear at the same simulated time. When several processes are ready, the second processor removes part of the queueing delay that existed with a single CPU.



(a) FIFO



(b) Round Robin, $q = 4$



(c) MLFQ, $\text{boost} = 20$, $\text{quantum} = 2$

Figure 25: Scheduling timelines for Process 5, Part 2. The horizontal axis is simulated time; each coloured interval represents CPU execution by a process.

9. Comparing the two parts visually

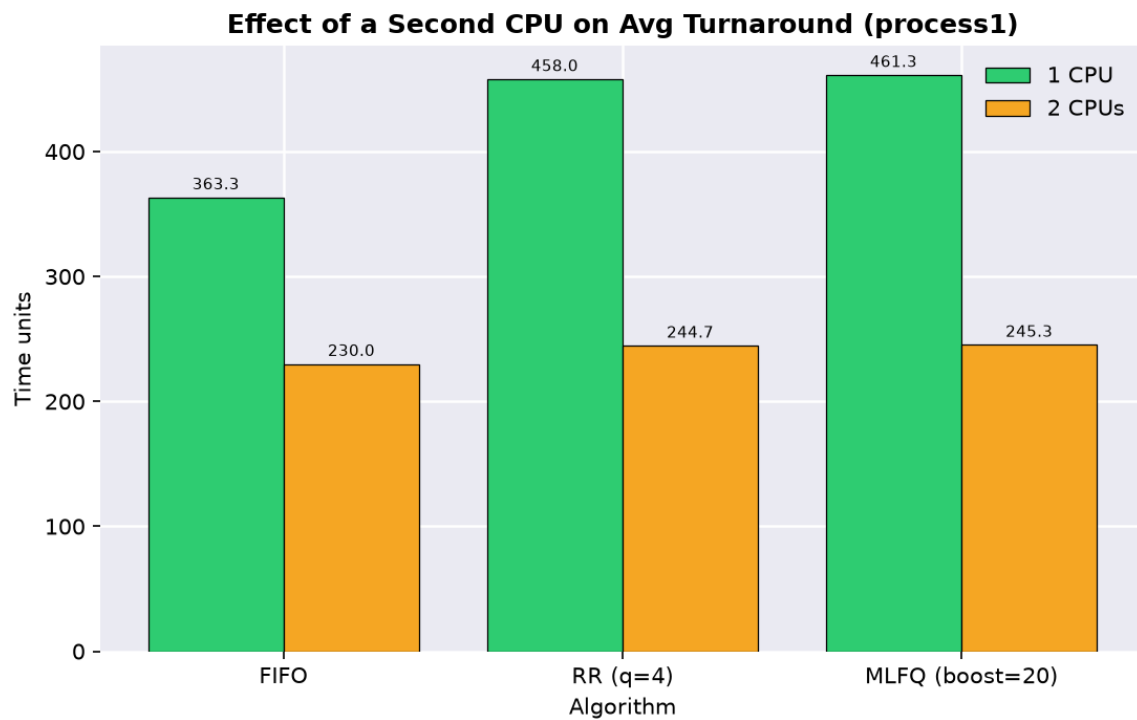


Figure 26: P1: average turnaround with one versus two CPUs.

P1 shows a large reduction because its processes are CPU-bound and can genuinely overlap.

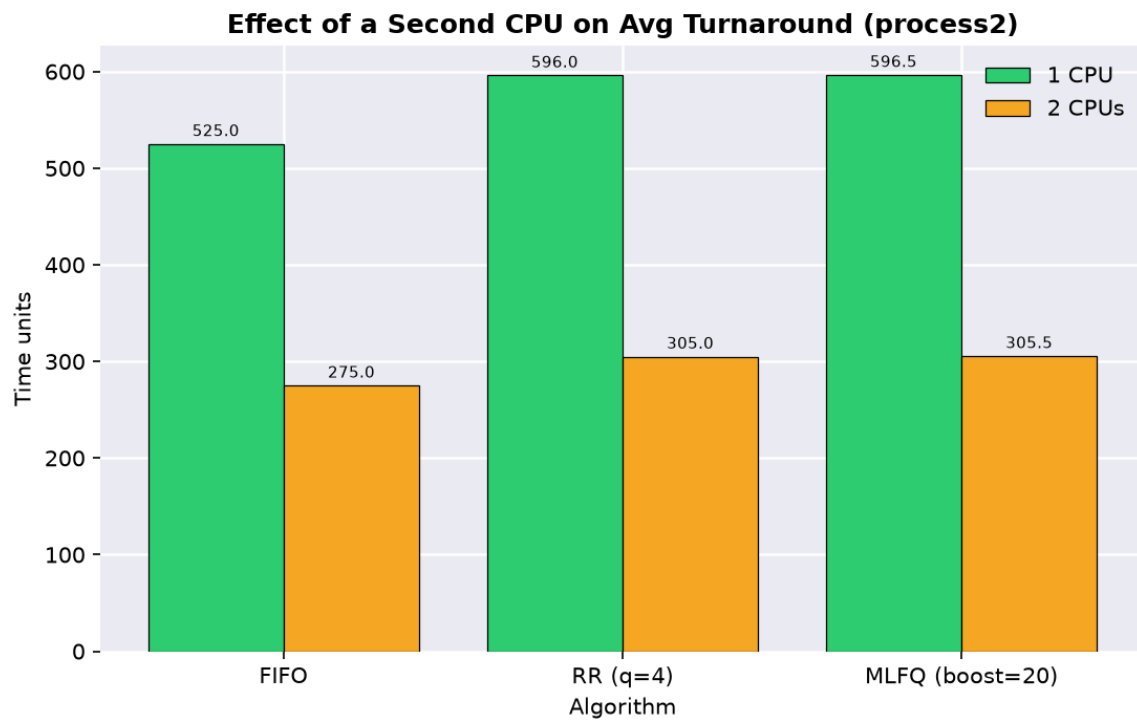


Figure 27: P2: average turnaround with one versus two CPUs.

P2 also benefits strongly. Its processes have repeated CPU phases, so whenever more than one process

is ready, the second CPU can execute useful work rather than leaving it queued.

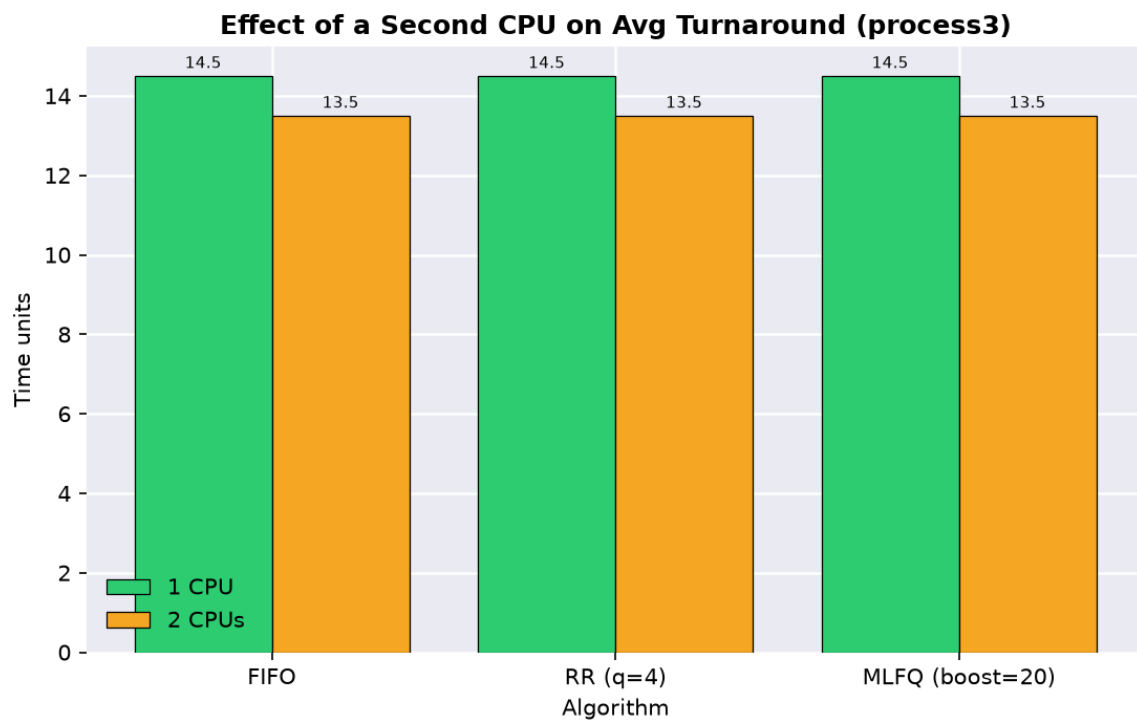


Figure 28: P3: average turnaround with one versus two CPUs.

P3 is the opposite case. The tiny reduction is consistent with its I/O-dominated structure.

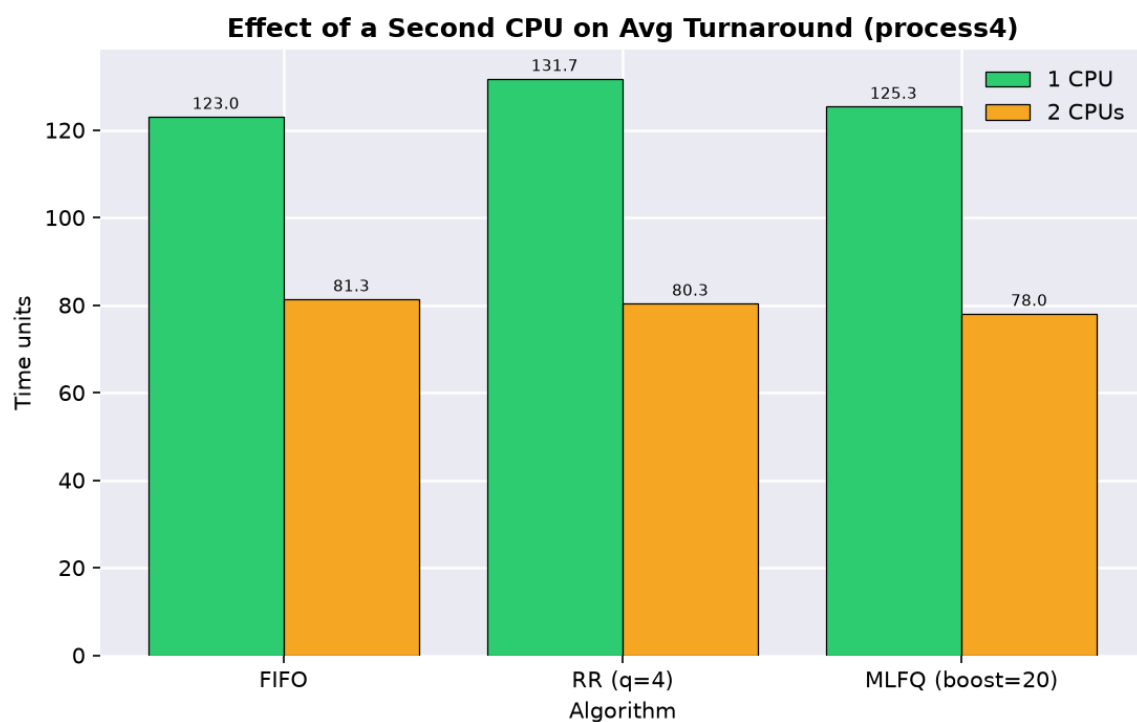


Figure 29: P4: average turnaround with one versus two CPUs.

P4 contains enough independent CPU work to keep the extra processor useful, particularly because several processes repeatedly return from I/O.

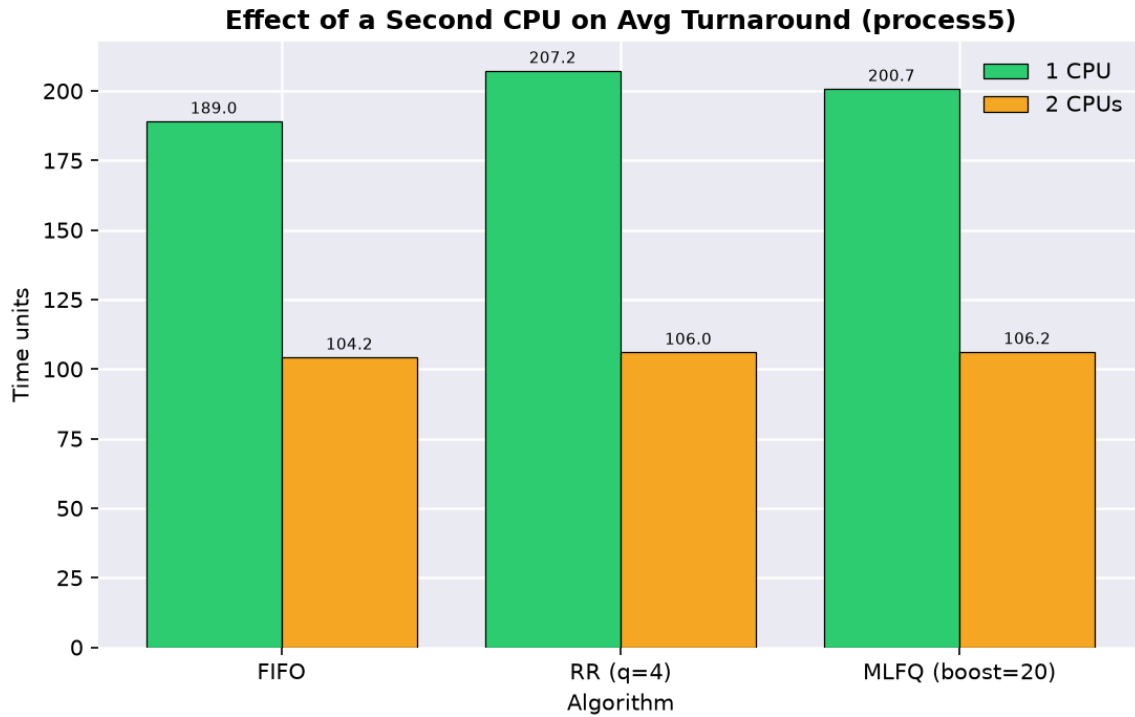


Figure 30: P5: average turnaround with one versus two CPUs.

P5 shows the same effect strongly: the extra processor allows the CPU-intensive and bursty processes to overlap instead of forcing them through one execution slot.

10. Simulator runtime

The simulator runtime is reported in milliseconds by the program. The values are extremely small for these workloads, typically around a few hundredths of a millisecond.

These numbers should be interpreted carefully. They measure the wall-clock time required for our C++ simulator to perform the simulation, not the simulated turnaround time of the processes. At this scale, differences of a few hundredths of a millisecond can also be affected by normal system timing noise.

For that reason, the runtime is useful as a required implementation metric, but it would be misleading to make strong performance claims about the scheduling algorithms from these tiny runtime differences alone.

The runtime measurements are nevertheless useful for checking that the simulation itself remains lightweight even when we introduce more scheduling bookkeeping for RR and MLFQ.

11. Putting the results together

The complete single-CPU results for the three main configurations are:

Process	FIFO		RR ($q = 4$)		MLFQ (boost 20)	
	Avg	Max	Avg	Max	Avg	Max
1	363.33	590	458.00	590	461.33	590
2	525.00	600	596.00	600	596.50	600
3	14.50	16	14.50	16	14.50	16
4	123.00	163	131.67	142	125.33	146
5	189.00	225	207.17	235	200.67	235

A few patterns stand out.

P1: FIFO has a lower average turnaround than RR/MLFQ in this particular workload because there are only three CPU-bound processes and no I/O. The preemption introduced by RR/MLFQ changes completion order without providing an I/O-related opportunity to hide waiting.

P2: the repeated CPU–I/O structure makes the ready queue dynamic, but the processes are similar enough that RR and MLFQ remain fairly close to one another in average turnaround.

P3: all the main policies are effectively constrained by the same thing: 1-unit CPU bursts surrounded by 5-unit I/O bursts. This is why many parameter changes have no visible effect.

P4/P5: these workloads provide the clearest evidence that scheduler parameters interact with workload structure. They have enough variation in CPU burst length and I/O behaviour for quantum, allotment and boost choices to change the execution order.

12. Final observations

The experiment makes several points that are easy to miss when looking only at textbook definitions.

1. **The workload matters as much as the scheduling rule.** The same RR implementation produces a completely flat quantum curve for P3 but a noticeably varying curve for P5.
2. **A larger or smaller quantum is not intrinsically better.** The useful question is how the quantum relates to the CPU-burst lengths and to the times at which processes return from I/O.
3. **MLFQ priority boosting is a scheduling mechanism, not a guaranteed turnaround improvement.** Its effect is positive for some workloads, neutral for others, and slightly increases average turnaround for P2 in our experiment.
4. **Average and maximum turnaround describe different parts of the workload.** P4’s MLFQ boost reduces average TAT while increasing maximum TAT by one time unit.
5. **Two processors help when CPU work can overlap.** The large reductions for P1, P2, P4 and P5 contrast with the very small change for I/O-dominated P3.
6. **The Gantt charts explain the numbers.** FIFO produces long uninterrupted regions, RR produces repeated time slices, and MLFQ produces changing priority levels and corresponding changes in execution order.

Overall, the useful outcome of the lab is not a single ordering of the algorithms. It is seeing how scheduling decisions propagate through an actual workload: arrivals determine who enters the ready queue, CPU bursts determine how long a process can occupy the processor, I/O determines when work returns, and the scheduling policy determines which ready process gets the next opportunity to execute. The graphs make that interaction visible.